



Escuela
Politécnica
Superior

Aplicación móvil para agrupaciones musicales



Grado en Ingeniería Informática

Trabajo Fin de Grado

Autor:

Ismael Ponce Navarro

Tutor/es:

David Corredor Lacha

Junio 2025



Universitat d'Alacant
Universidad de Alicante

1. Justificación y objetivos

La idea de este proyecto surgió durante el verano de 2024, yo soy trompetista y formo parte de la directiva de una banda de música y de una charanga. En mi día a día como músico y responsable, me di cuenta de que la forma de conseguir refuerzos para completar las plantillas era poco efectiva, ya que teníamos que ir preguntando a contactos de otros pueblos si estaban disponibles para tocar con nosotros, por lo que es complicado encontrar gente disponible, y pensé que podría hacer una aplicación con la que contactar con todos los músicos de una zona sin recurrir a escribirles un mensaje por WhatsApp.

Además, al formar parte de una charanga, conozco la forma en la que los clientes contactan con grupos como nosotros, y tiene margen de mejora, por lo que pensé que podría facilitar también la forma en la que el público encuentra charangas disponibles en su zona y hacer más fácil conseguir nuevas actuaciones.

Por estos motivos, me planteo la idea de desarrollar una aplicación fiable e intuitiva tanto para músicos como para clientes, con el fin de ayudar el proceso de la gestión de un acto musical, desde su contratación hasta la parte de conformar la plantilla de músicos.

2. Agradecimientos

A mi familia, por haberme aguantado y apoyado durante estos años, especialmente a mi madre Amparo y mi hermana María Amparo, que me aconsejaron no rendirme en los peores momentos, sin ellos no podría haber conseguido esto.

A mi grupo de amigos de Caudete, con los que he vivido muchas experiencias y me han ayudado durante años.

A mis amigos de Alicante, a los que he conocido durante esta etapa universitaria y he disfrutado de grandes momentos con ellos. Especialmente Juanvi y Nerea, con los que he convivido en el piso durante años y me han apoyado y ayudado de infinitas formas, y a Samuel, que aunque lo conocí más tarde, también ha sido un gran amigo.

3. Índice

1. Justificación y objetivos	2
2. Agradecimientos	3
3. Índice	4
4. Índice de tablas	7
5. Índice de figuras	8
6. Introducción.....	10
6.1. Resumen.....	10
6.2. Objetivos.....	12
6.3. Estudio de mercado.....	13
6.3.1. Charangas.info.....	13
6.3.2. Charangas.com	14
6.4. Análisis DAFO	15
7. Herramientas y tecnologías	16
7.1. MySQL	16
7.2. Backend: Node.js y Express	17
7.3. Frontend: Angular.....	18
7.4. Visual Studio Code	19
7.5. Azure.....	20
7.6. Git y GitHub	22
7.7. Jira.....	22
7.8. ChatGPT + DALL-E.....	23
8. Metodologías aplicadas	24
8.1. Tablero Kanban Jira.....	25
8.2. Github	26
9. Implementación	27

9.1. Base de datos MySQL	27
9.2. Back-end de la aplicación (Node.js y Express)	29
9.2.1. Rutas relacionadas con los usuarios	30
9.2.2. Rutas de utilidad.....	30
9.2.3. Rutas relacionadas con las charangas.....	31
9.2.4. Rutas relacionadas con los músicos	32
9.2.5. Rutas relacionadas con las ofertas de clientes para charangas.....	32
9.2.6. Rutas relacionadas con los actos de charangas para músicos	35
9.2.7. Autenticación y seguridad de la aplicación.....	37
9.3. Front-end de la aplicación (Angular).....	39
9.3.1. Barra de navegación.....	40
9.3.2. Vista inicio	42
9.3.3. Vista Iniciar sesión	43
9.3.4. Vista Registro.....	44
9.3.5. Vista ¿Quiénes somos?.....	46
9.3.6. Vista Tratamiento de datos.....	47
9.3.7. Vista Perfil.....	48
9.3.8. Vista Nuestras Charangas.....	50
9.3.9. Vista Ofertas para Charangas	51
9.3.10. Vista Búsqueda de músicos.....	56
9.3.11. Vista Anuncios Charangas	61
9.3.12. Vista Anuncios Músicos.....	63
9.4. Procedimientos con mayor complejidad.....	65
9.4.1. Vista ofertas para charangas.....	65
9.4.2. Vista búsqueda de músicos	67
9.5. Despliegue de la aplicación	68
9.5.1. Despliegue Base de datos con Servidor flexible for MySQL de Azure.....	68

9.5.2. Despliegue Backend con App Service de Azure.....	68
9.5.3. Despliegue Frontend con Aplicación Web Estática de Azure	69
9.5.4. Dominio de la aplicación.....	71
9.5.5. Usuarios para pruebas	72
10. Conclusiones.....	73
10.1. Resultados.....	73
10.2. Posibles mejoras	73
11. Referencias	75

4. Índice de tablas

Tabla 1. Comparativa funcionalidades aplicaciones	14
Tabla 2. Análisis DAFO	15

5. Índice de figuras

Figura 1. Logotipo de la aplicación	23
Figura 2. Tablero Kanban en Jira	25
Figura 3. Diagrama entidad-relación	28
Figura 4. Estructura del Back-end	29
Figura 5. Estructura del frontend del proyecto	39
Figura 6. Barra de navegación para usuario sin iniciar sesión.	40
Figura 7. Barra de navegación para usuario músico.....	41
Figura 8. Barra de navegación para usuario charanga.....	41
Figura 9. Barra de navegación para usuario cliente.....	41
Figura 10. Vista inicio	42
Figura 11. Vista iniciar sesión	43
Figura 12. Vista registro parte 1	45
Figura 13. Vista registro parte 2	45
Figura 14. Vista ¿Quiénes somos?	46
Figura 15. Vista Tratamiento de datos	47
Figura 16. Vista perfil.....	48
Figura 17. Vista editar perfil.....	49
Figura 18. Vista Listado Charangas	50
Figura 19. Vista Ofertas para charangas, sección Crear oferta.	53
Figura 20. Vista Ofertas para charangas, sección listado de ofertas del cliente.....	53
Figura 21. Vista Ofertas para charangas, sección solicitudes recibidas	54
Figura 22. Vista Ofertas para charangas, sección solicitud aceptada.....	54
Figura 23. Vista Ofertas para charangas, ejemplo de correo.....	55
Figura 24. Vista búsqueda músicos, crear acto.	57
Figura 25. Vista búsqueda músicos, listado actos.	58
Figura 26. Vista búsqueda músicos, solicitudes recibidas.	58
Figura 27. Búsqueda músicos, músicos contratados.	59
Figura 28. Búsqueda músicos, ejemplo correo.....	59
Figura 29. Búsqueda músicos, vista del músico.....	60
Figura 30. Anuncios charangas, crear anuncio.....	61
Figura 31. Anuncios charangas, listado anuncios.	62

Figura 32. Anuncios músicos, crear anuncio.....	64
Figura 33. Anuncios músicos, listado anuncios.	64
Figura 34. Parámetros conexión con la Base de Datos.....	68
Figura 35. Centro implementación App Service Azure	69
Figura 36. Despliegues para producción GitHub	70
Figura 37. Ejemplo aplicación corriendo	71

6. Introducción

6.1. Resumen

El objetivo principal de este Trabajo de Fin de Grado es crear una aplicación móvil llamada **EasyMusic**, para facilitar el día a día de las charangas, los músicos y de los clientes. Una charanga (Arlequin Band Street, s.f.) es un grupo de música formado por instrumentos de viento y percusión que se encarga de animar por las calles determinados festejos como desfiles, bodas, fallas, etc.

Por un lado, se facilita el papel del cliente ofreciendo un catálogo de charangas y pudiendo buscar por provincias. Por otro lado, se facilita el papel de las charangas, ya que en la actualidad cuando a un grupo le faltan músicos para una actuación no hay ninguna aplicación que ayude a encontrar músicos, y normalmente hay que recurrir a tirar de contactos para completar la plantilla. Además las charangas podrán encontrar nuevos actos más fácil, ya que se facilita su contacto con nuevos clientes.

En cuanto a los músicos, también podrán encontrar grupos musicales con los que tocar de manera más sencilla.

EasyMusic ha sido creada de manera fácil e intuitiva para que sea accesible por cualquier individuo, ya que el rango de edad de los músicos y de los clientes abarca desde adolescentes hasta gente de 70 años.

EasyMusic cuenta con 3 tipos de usuario (Cliente, Charanga y Músico), para ayudar a cada parte implicada en un acto musical. Las principales funcionalidades de cada tipo de usuario son las siguientes.

- Usuario cliente:
 - Pueden publicar ofertas buscando una charanga para un determinado acto.
 - Pueden contactar con las charangas para consultar su disponibilidad.
 - Pueden ver los anuncios que publican las charangas.
- Usuario charanga:

- Pueden solicitar participar en las ofertas que publican los clientes.
 - Anunciar sus servicios para festividades concretas.
 - Pueden publicar actos en los que especifican qué número de músicos y qué instrumentos necesitan para poder completar su plantilla.
 - También pueden consultar los anuncios que publican los músicos ofreciendo su disponibilidad.
- Usuario músico:
 - Pueden inscribirse en los actos que publican las charangas buscando músicos.
 - Además pueden publicar anuncios ofreciendo su disponibilidad.

Además, los usuarios pueden aplicar filtros por provincia a la hora de buscar actos o charangas, y en el caso de los músicos también se puede filtrar por instrumento. Esto facilita el trabajo a la hora de encontrar el usuario deseado.

Una vez un acto se haya llevado a cabo, existe la opción de valorar a la Charanga y al músico, para que de esta manera estos usuarios tengan una valoración de cara a futuros contratos. Además, las charangas también pueden valorar al cliente una vez ha acabado el acto, para que las demás charangas sepan si ese cliente ofrece un buen trato a los grupos que contrata.

6.2. Objetivos

Los objetivos que se pretenden alcanzar con esta aplicación son los siguientes:

- Desarrollar una aplicación fácil e intuitiva para usuarios de cualquier grupo de edad, ya que tanto los clientes como los músicos, pueden ser desde adolescentes hasta jubilados.
- Ofrecer una solución que permita a los clientes encontrar una charanga de forma rápida.
- Proporcionar a las charangas una forma sencilla de encontrar refuerzos para sus actuaciones.
- Proporcionar a los músicos una forma de encontrar grupos musicales con los que hacer actuaciones.

Con estos objetivos se pretende hacer que la experiencia de las personas involucradas en eventos musicales sea más satisfactoria y sencilla, ahorrando tiempo y aportando soluciones a las personas de forma gratuita.

6.3. Estudio de mercado

En el mercado actualmente hay algunas páginas que ofrecen información para contactar con charangas. En ellas las charangas se registran y los clientes pueden buscar información sobre ellas y contactar con ellos, sin embargo, esas aplicaciones no cuentan con funcionalidades para buscar músicos para completar plantillas, por lo que este aspecto es una fortaleza de mi aplicación frente a la competencia.

Alguna de las aplicaciones que se han estudiado son las siguientes:

6.3.1. Charangas.info

Charangas.info (Charangas.info, s.f.) es una página web en la que las charangas se pueden publicitar. Para ello los grupos pagan una cuota anual y pueden promocionarse en la página, una vez se les acaba la cuota, dejan de aparecer en el buscador.

En ella puedes buscar charangas por zona y disponen de un mapa con las ubicaciones marcadas de las charangas que se anuncian.

Además ofrecen cuotas para diseñar páginas web a charangas o para arreglistas que quieren promocionarse.

Puntos a favor:

- Es una página bastante conocida dentro del mundo de las charangas.
- Disponen de buscador por provincia para encontrar charangas cercanas.
- Ofrecen servicios para crear páginas webs a particulares.

Puntos en contra:

- Es necesario tener una cuota activa para anunciarte en ella.
- No ofrecen soluciones a grupos recién empezados, ya que cuando nace una charanga suele ser formada por adolescentes, y no disponen de capital para pagar por promociones.

6.3.2. Charangas.com

Charangas.com (charangas.com, s.f.) una página web creada a finales de 2024, en la que se prestan servicios para promocionar charangas y gestionar los contratos de las actuaciones. En ella las charangas se pueden promocionar y los clientes pueden buscar charanga por provincia. Esta página destaca en ofrecer ayuda al cliente a la hora de realizar la contratación, ofreciendo información sobre el proceso y gestionando los presupuestos.

Puntos a favor:

- Ofrecen promocionarse de manera gratuita a las charangas que se registran en su página, facilitando que los clientes puedan encontrarlos.
- Ofrecen ayuda a los clientes a la hora de encontrar una charanga adecuada para sus actuaciones y a gestionar el presupuesto del contrato.

Puntos en contra:

- La página web se lleva una parte del dinero del contrato.

Una vez analizadas las aplicaciones que hay en el mercado, voy a resumir los puntos clave para mostrar una comparación de las funcionalidades que ofrece EasyMusic frente a la competencia en el mercado.

Aplicación	Promoción gratuita	Búsqueda de ofertas de clientes por provincia	Búsqueda de charangas por provincia	Búsqueda de músicos por provincia o instrumento	Posibilidad de completar plantillas de músicos para un acto
Charangas.info	✗	✗	✓	✗	✗
Charangas.com	✓	✗	✓	✗	✗
EasyMusic	✓	✓	✓	✓	✓

Tabla 1. Comparativa funcionalidades aplicaciones

6.4. Análisis DAFO

Se trata de una metodología de análisis que permite conocer los factores externos e internos que impactan en el éxito o viabilidad de un proyecto. Sus siglas significan (Debilidades, Amenazas, Fortalezas y Oportunidades), siendo factores internos las Debilidades y Fortalezas, y factores externos las Amenazas y Oportunidades.

Despues de haber hecho el estudio de mercado, realizamos el análisis DAFO (Unir, s.f.) de la aplicación:

	Análisis interno	Análisis externo
Negativos	<u>D</u>ebilidad No dispongo de clientes. Cuento con presupuesto y tiempo de desarrollo limitado.	<u>A</u>menaza Existen aplicaciones bastante conocidas en el sector. Las aplicaciones existentes cuentan con más experiencia.
Positivos	<u>F</u>ortaleza Conozco bien los puntos a mejorar en la contratación de músicos. Conozco a músicos de distintas zonas y puedo dar a conocer mi aplicación rápidamente.	<u>O</u>portunidad Las aplicaciones del mercado solo se enfocan en promocionar a las charangas. Aprovechar la funcionalidad de completar plantillas y buscar ofertas para vender mi idea frente a las demás.

Tabla 2. Análisis DAFO

7. Herramientas y tecnologías

7.1. MySQL

MySQL (OpenWebinars, s.f.) es el sistema de gestión de bases de datos relacional más extendido en la actualidad al estar basada en código abierto. Presenta algunas ventajas que lo hacen muy interesante para los desarrolladores. La más evidente es que trabaja con bases de datos relacionales, es decir, utiliza tablas múltiples que se interconectan entre sí para almacenar la información y organizarla correctamente.

Sus principales características son:

- **Arquitectura Cliente y Servidor:** MySQL basa su funcionamiento en un modelo cliente y servidor. Es decir, clientes y servidores se comunican entre sí de manera diferenciada para un mejor rendimiento.
- **Compatibilidad con SQL:** SQL es un lenguaje generalizado dentro de la industria. Al ser un estándar MySQL ofrece plena compatibilidad por lo que si has trabajado en otro motor de bases de datos no tendrás problemas en migrar a MySQL.
- **Vistas:** Desde la versión 5.0 de MySQL se ofrece compatibilidad para poder configurar vistas personalizadas del mismo modo que podemos hacerlo en otras bases de datos SQL. En bases de datos de gran tamaño las vistas se hacen un recurso imprescindible.
- **Procedimientos almacenados.** MySQL posee la característica de no procesar las tablas directamente sino que a través de procedimientos almacenados es posible incrementar la eficacia de nuestra implementación.
- **Desencadenantes.** MySQL permite además poder automatizar ciertas tareas dentro de nuestra base de datos. En el momento que se produce un evento otro es lanzado para actualizar registros u optimizar su funcionalidad.

Por todo esto, se ha elegido MySQL para gestionar las bases de datos del proyecto y el software MySQL Workbench para realizar el diseño e implementación de la base de datos.

7.2. Backend: Node.js y Express

Express (kinsta, s.f.) es un framework de backend basado en Node.js (Node.js, s.f.) minimalista y rápido, que proporciona características y herramientas robustas para desarrollar aplicaciones de backend escalables. Ofrece un sistema de enrutamiento y características simplificadas para ampliar el framework con componentes y partes más potentes en función de los casos de uso de tu aplicación.

Se ha elegido este framework para desarrollar el backend de esta aplicación ya que presenta numerosas ventajas:

- Express proporciona un rendimiento óptimo en el procesamiento de peticiones y respuesta, lo que hace que la aplicación sea más rápida.
- Node.js emplea una arquitectura basada en eventos y un modelo de Entrada/Salida. Esto permite manejar múltiples solicitudes de manera concurrente, mejorando la capacidad de respuesta y la escalabilidad.
- Express proporciona un conjunto de funciones que permiten crear API REST de manera sencilla. De esta manera se facilita la gestión de solicitudes y respuestas HTTP, lo cual es fundamental para la comunicación entre el frontend y el backend de la aplicación.

7.3. Frontend: Angular

Angular (Gonçalves, s.f.) es un Framework de JavaScript de código abierto escrito en TypeScript. Su objetivo principal es desarrollar aplicaciones de una sola página.

TypeScript define un conjunto de tipos de JavaScript, lo que ayuda a los usuarios a escribir código JavaScript que es más fácil de entender. Todo el código TypeScript se compila con JavaScript y se puede ejecutar sin problemas en cualquier plataforma. TypeScript no es obligatorio para desarrollar una aplicación Angular. Sin embargo, es muy recomendable ya que ofrece una mejor estructura sintáctica, al tiempo que hace que la base de código sea más fácil de entender y mantener.

Angular (Axarnet, s.f.) sigue el patrón de diseño Modelo-Vista-Controlador (MVC), que separa claramente la lógica de presentación de los datos y la interacción del usuario.

El núcleo de Angular se basa en módulos, componentes y servicios. Los módulos son contenedores que agrupan componentes relacionados y otros recursos. Los componentes son bloques de construcción fundamentales en Angular y representan diferentes partes de la interfaz de usuario. Los servicios proporcionan funcionalidades compartidas y se utilizan para realizar tareas como el manejo de datos o la comunicación con servidores.

Angular utiliza un sistema de enlace de datos bidireccional, lo que significa que cualquier cambio en los datos se reflejará automáticamente en la interfaz de usuario y viceversa. Esto simplifica enormemente el desarrollo de aplicaciones interactivas y en tiempo real.

7.4. Visual Studio Code

Visual Studio Code (VS Code) (Cimas Cuadrado, s.f.) es un editor de código fuente desarrollado por Microsoft, diseñado para ser ligero, rápido y altamente personalizable. A diferencia de otros editores de código, como los IDE completos (Entornos de Desarrollo Integrados), VS Code se enfoca en ofrecer una experiencia ágil para la escritura y edición de código, sin sacrificar características avanzadas.

Lo que lo diferencia es su capacidad de ser extendido con una gran cantidad de extensiones y herramientas, adaptándose a las necesidades específicas de cada desarrollador.

Entre sus características clave, se incluyen la autocompletado inteligente mediante IntelliSense, la depuración integrada, y el control de versiones con Git, lo que convierte a VS Code en una opción versátil tanto para desarrolladores individuales como para equipos.

Uno de los mayores puntos a favor de Visual Studio Code es que es completamente gratuito y multiplataforma. Esto significa que puedes descargar y utilizar VS Code en Windows, macOS y Linux sin coste alguno.

Gracias a su gran cantidad de herramientas y extensiones, el desarrollo de la aplicación ha sido más efectivo, ya que empleando sus extensiones de MySQL y Git, permiten acceder a la base de datos MySQL directamente en el editor, y a realizar los cambios en GitHub.

7.5. Azure

Para el despliegue de la aplicación en un sitio web se han empleado las herramientas que ofrece Azure Portal (Azure, s.f.). Azure Portal es una consola unificada basada en web que permite crear y administrar todos los recursos de Azure. Con Azure Portal, puede administrar su suscripción a Azure mediante una interfaz gráfica de usuario. Puede compilar, administrar y supervisar todo, desde aplicaciones web sencillas hasta implementaciones complejas en la nube en el portal.

Por un lado he utilizado un Servidor Flexible de Azure Database for MySQL, (Microsoft, s.f.) en el que he cargado mi base de datos. Azure Database for MySQL (servidor flexible) es un servicio de base de datos relacional listo para la producción totalmente administrado de Microsoft Cloud. El servidor le proporciona un control y una flexibilidad más pormenorizados sobre las funciones y la configuración de la administración de la base de datos.

Por otro lado, el backend lo he desplegado usando un App Service (Microsoft, s.f.). Azure App Service es una plataforma que le permite ejecutar aplicaciones web, back-ends móviles y API REST sin preocuparse por administrar la infraestructura subyacente. Piense en él como un potente servicio de hospedaje web que se encarga de todo el trabajo pesado para usted, por lo que puede centrarse en la creación de excelentes aplicaciones.

App Service admite una variedad de pilas web: .NET, Java, Node.js, Python y PHP, y puede ejecutarlas en Windows y Linux. O bien, si la aplicación está en contenedor, puede implementarla como un contenedor personalizado. En nuestro caso elegimos la pila de Node.js.

Finalmente el Frontend, lo he desplegado con una Aplicación Web Estática (Microsoft, s.f.). Azure Static Web Apps es un servicio que implementa automáticamente aplicaciones web de pila completa en Azure desde un repositorio de código. El flujo de trabajo de Azure Static Web Apps se adapta al flujo de trabajo diario de un desarrollador. Las aplicaciones se compilan e implementan en función de los cambios en el código.

Cuando se crea una aplicación web estática, Azure interactúa directamente con GitHub o para supervisar una rama de su elección. Cada vez que inserte confirmaciones o acepte solicitudes de incorporación de cambios en la rama inspeccionada, se ejecuta automáticamente una compilación y la aplicación y la API se implementan en Azure.

Las aplicaciones web estáticas se suelen crear con bibliotecas y marcos web como Angular, React, Svelte, Vue o Blazor, en los que no se requiere representación en el servidor. Estas aplicaciones incluyen recursos HTML, CSS, JavaScript y de imagen que componen la aplicación. Con un servidor web tradicional, estos recursos se ofrecen desde un único servidor junto con los puntos de conexión de la API necesarios.

De esta manera he conseguido que la aplicación esté funcionando de manera online en el siguiente sitio web: <https://victorious-stone-011abec10.6.azurestaticapps.net/>

El detalle de como se ha implementado la aplicación en Azure será detallado al final del documento.

7.6. Git y GitHub

Git es un sistema de control de versiones de código fuente. Es una herramienta que se utiliza para llevar un registro de los cambios en el código fuente de un proyecto y permitir a varios desarrolladores trabajar en el mismo proyecto de manera simultánea.

Con Git, es posible crear un repositorio que contenga el código fuente de un proyecto y llevar un registro de todos los cambios realizados en el repositorio. Los desarrolladores pueden colaborar en el mismo proyecto al hacer «commits» (guardar cambios) en el repositorio y luego hacer «push» (enviar) estos cambios a un servidor centralizado (en nuestro caso GitHub) para que puedan ser utilizados por otros desarrolladores.

GitHub (B, s.f.) es un servicio basado en la nube que aloja un sistema de control de versiones llamado Git. Gracias a GitHub creamos un directorio en la nube en la que ir guardando las versiones de nuestro proyecto y mantener un control seguro de los cambios.

7.7. Jira

El software que se ha empleado para realizar la planificación del proyecto ha sido Jira, ya que nos permite realizar una buena gestión de las tareas mediante un tablero Kanban (Atlassian, s.f.).

Un tablero de Kanban es una herramienta ágil de gestión de proyectos diseñada para ayudar a visualizar el trabajo, limitar el trabajo en curso y maximizar la eficiencia. Los tableros de Kanban utilizan tarjetas, columnas y la mejora continua para ayudar a los equipos tecnológicos y de servicios a comprometerse con la cantidad de trabajo adecuada y a llevarla a cabo.

7.8. ChatGPT + DALL-E

Para diseñar el logo de la aplicación se ha utilizado ChatGPT + DALL-E. DALL-E (Fernández, s.f.) es un sistema de inteligencia artificial creado por OpenAI. Es una IA que genera imágenes a partir de texto, de forma que tú le escribes lo que quieres ver mediante un prompt o comando textual, y la generará de la nada.

Esta inteligencia artificial está basada en los modelos de lenguaje como GPT. Gracias a esto, este sistema es capaz de entender lo que le pides con un lenguaje natural, ya que ha sido entrenada para entender la manera en las que solemos hablar y cómo nos expresamos para pedir o preguntar algo.

Para diseñar el logo, se le escribió un prompt a ChatGPT especificando el nombre de la aplicación, los colores que quería en el logo, pidiéndole que añadiera una trompeta y un bombo, y añadiendo un eslogan debajo, con lo que obtuve el siguiente logo.

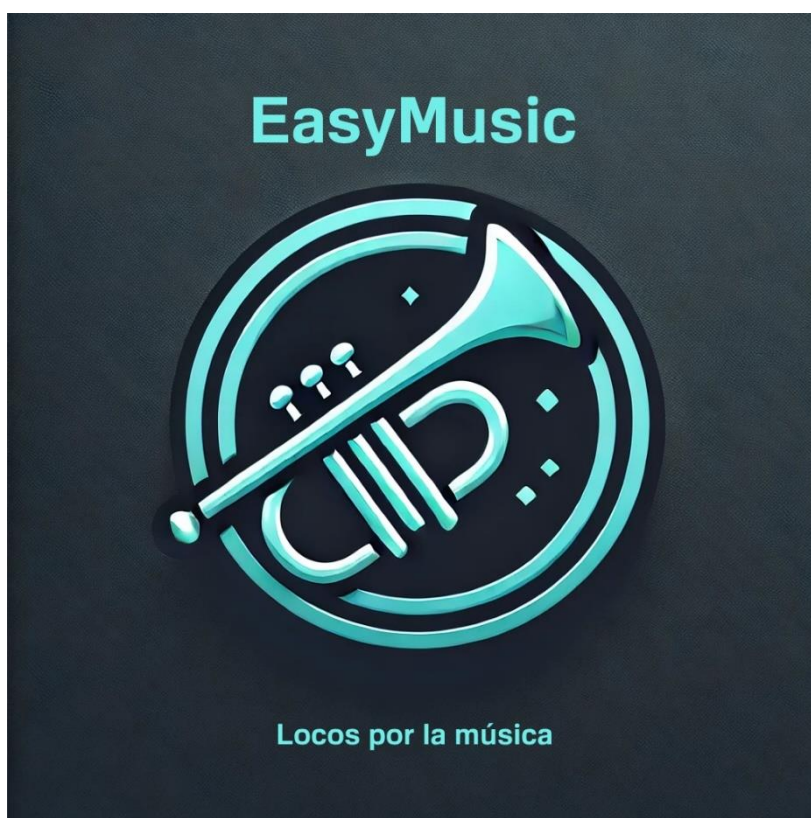


Figura 1. Logotipo de la aplicación

8. Metodologías aplicadas

Para organizar las tareas del proyecto, se ha optado por seguir las metodologías ágiles (Zendesk, s.f.). Son un conjunto de técnicas aplicadas en ciclos de trabajo cortos, con el objetivo de que el proceso de entrega de un proyecto sea más eficiente. Así, con cada etapa completada, ya se pueden entregar avances y se deja de lado la necesidad de esperar hasta el término del proyecto.

Las metodologías ágiles fomentan la autoorganización, ya que se divide el trabajo en tareas más pequeñas y manejables, fomentando la responsabilidad individual y optimizando la productividad.

Al tratarse de un proyecto individual, se ha optado por utilizar una metodología Kanban, usando tableros en Jira. Esta metodología es más adecuada frente a otras más populares como Scrum, ya que Scrum está más orientada al trabajo en equipo.

8.1. Tablero Kanban Jira

A la hora de organizar las tareas, he creado un tablero Kanban en Jira, distribuyendo las tareas en 3 columnas. De esta manera las tareas se organizan en 3 estados:

- **Por hacer:** Para aquellas tareas que tienen que completarse en el futuro.
- **En progreso:** Para las tareas que se están llevando a cabo en este momento.
- **Completados:** Para las tareas que ya han sido realizadas.

Este enfoque permite ver de manera rápida y visual el estado del proyecto y cuáles son las tareas que ya se han finalizado.

A continuación, se adjunta una captura del tablero Kanban tomada durante el desarrollo del proyecto.

Tablero KAN

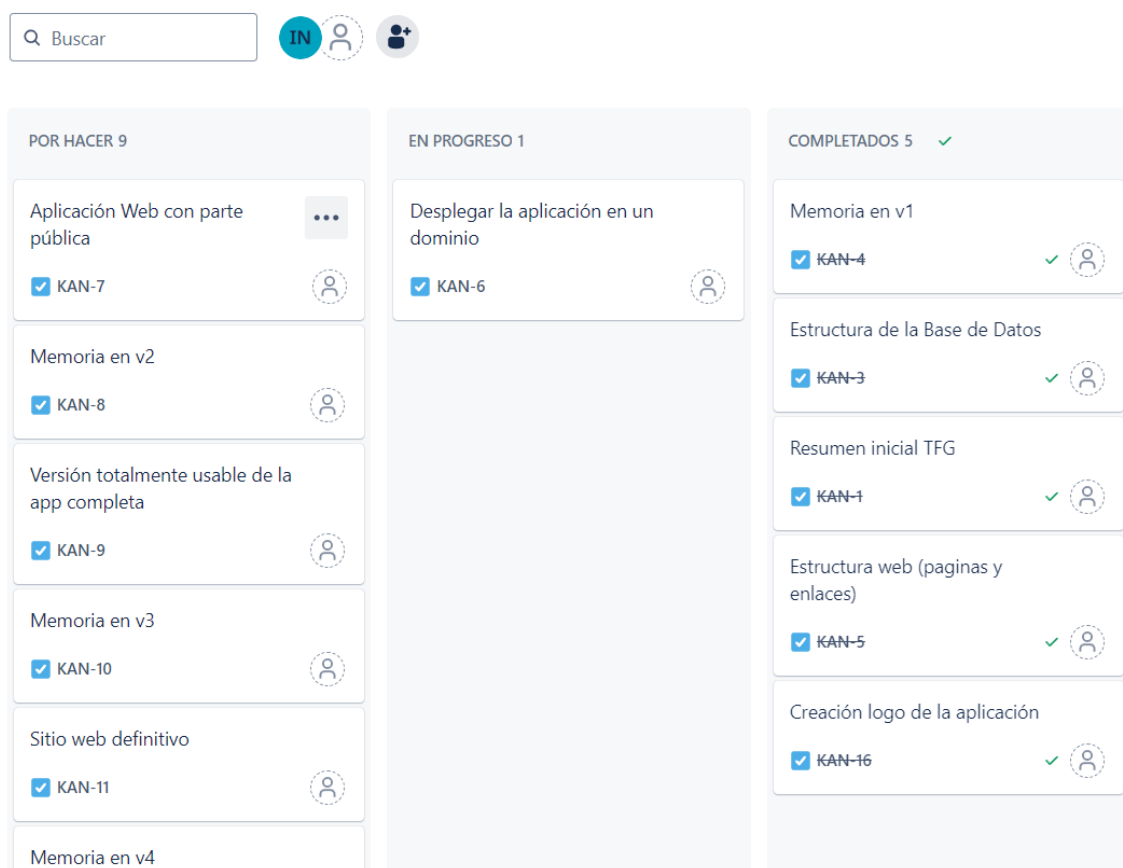


Figura 2. Tablero Kanban en Jira

8.2. Github

Para la organización del código del proyecto, se ha empleado un proyecto en GitHub. Para ello se creó un directorio llamado easymusic el cual aloja el código del proyecto. Después este código sirve para que Azure lance la aplicación y esté corriendo en internet.

Para distribuir las versiones del código se ha dividido en 2 ramas:

- **Rama Develop:** Contiene la versión estable del proyecto y la rama que Azure obtiene para lanzar la aplicación web.
- **Rama Ismael:** Contiene el código de versiones anteriores por si algún cambio produce errores en la aplicación estable.

De esta manera, logramos tener el código seguro ante pérdidas y podemos volver a versiones totalmente funcionales.

9. Implementación

9.1. Base de datos MySQL

El motor utilizado para la base de datos ha sido MySQL. La base de datos se ha construido utilizando MySQL Workbench, con el que utilizando su interfaz gráfica, ha facilitado el trabajo de creación de la base de datos. Una vez creado el diagrama, empleando la herramienta de Forward Engineer, la aplicación genera el código a partir del diagrama y crea la base de datos.

La base de datos cuenta con 14 tablas:

- **Usuarios:** Tabla que almacena los datos principales de los usuarios de la aplicación. Los usuarios se dividen en 3 distintos tipos.
- **Charangas:** Tabla que almacena los tipos de usuario Charanga.
- **Musicos:** Tabla que almacena los usuarios de tipo Musico.
- **Clientes:** Tabla que almacena los usuarios de tipo Cliente.
- **Valoraciones:** En esta tabla se almacenan todas las valoraciones que se realizan a los usuarios, para posteriormente generar la valoración media de cada usuario.
- **Provincia:** En esta tabla se almacenan las provincias de España.
- **Instrumento:** En esta tabla se almacenan los instrumentos que existen.
- **Ofertas:** Tabla que almacena los anuncios que publican los clientes cuando buscan una charanga.
- **Solicitudes:** Tabla que almacena las solicitudes que realizan las Charangas a las ofertas de los clientes.
- **Actos:** Tabla que almacena las publicaciones de las Charangas en las que buscan músicos para completar su plantilla.
- **Acto_instrumentos:** En esta tabla se almacenan los instrumentos que se buscan para un determinado acto.
- **Solicitudes_musicos:** Tabla que almacena las solicitudes que realizan los músicos a los actos que publican las charangas.
- **Musicos_contratados:** Cuando los musicos participan en un acto con una Charanga completando su plantilla se almacena en esta tabla y se asigna una valoración a su rendimiento.

- **Anuncios:** Tabla que almacena la información de los anuncios que ponen los músicos o las charangas para ofrecer sus servicios. Como estos anuncios solo pueden ser publicados por músicos o charangas, esta tabla se divide en 2 tipos.
- **Anuncios_charangas:** Tabla que almacena los anuncios de las charangas.
- **Anuncios_musicos:** Tabla que almacena los anuncios de los músicos.

La implementación de estas tablas ha dado lugar al siguiente diagrama entidad-relación:

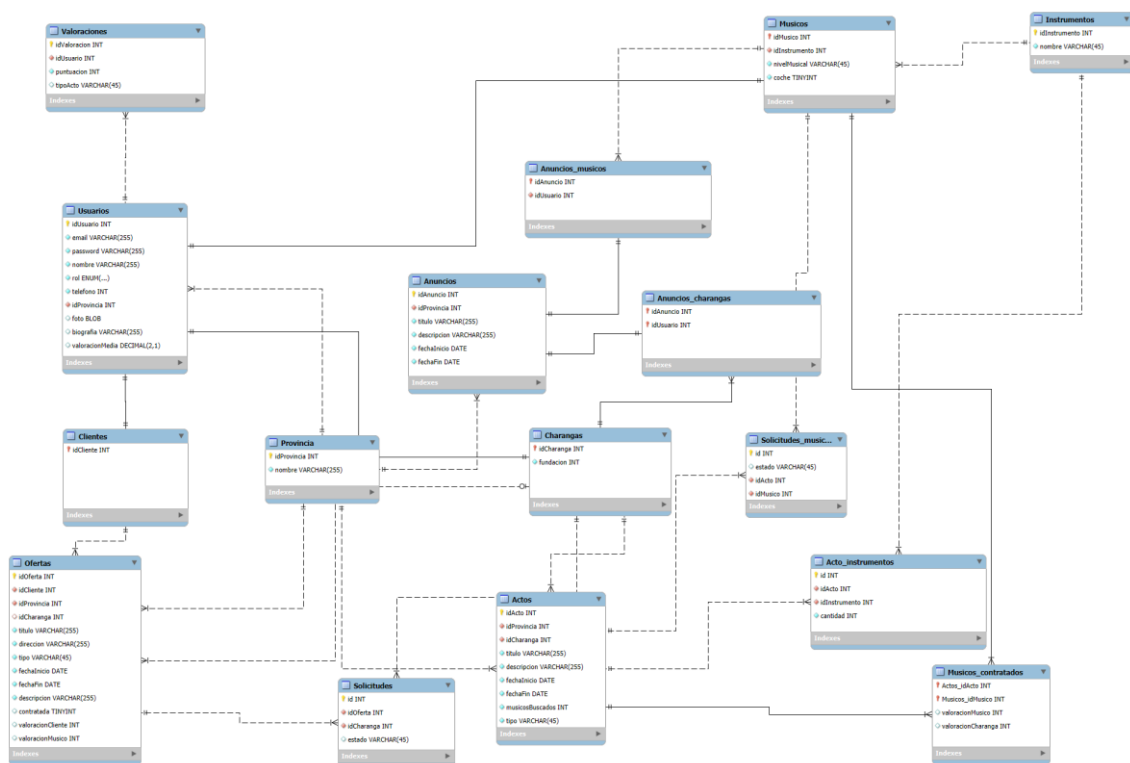


Figura 3. Diagrama entidad-relación

9.2. Back-end de la aplicación (Node.js y Express)

Para el desarrollo del back-end he empleado Node.js y Express. Para comunicarse correctamente con el Frontend he empleado Cors. Cors permite el intercambio de recursos entre diferentes dominios, lo cual es muy útil para aplicaciones en las que se utiliza arquitectura cliente-servidor, en la que el cliente está en un dominio diferente al servidor.

El código del backend está almacenado en el fichero server.js, en él se especifican todos los detalles para la conexión con la base de datos y la conexión con el frontend. Los demás ficheros de la carpeta de backend fueron generados para configurar la conexión con el frontend al subir el backend a Azure. El despliegue en Azure será comentado posteriormente.

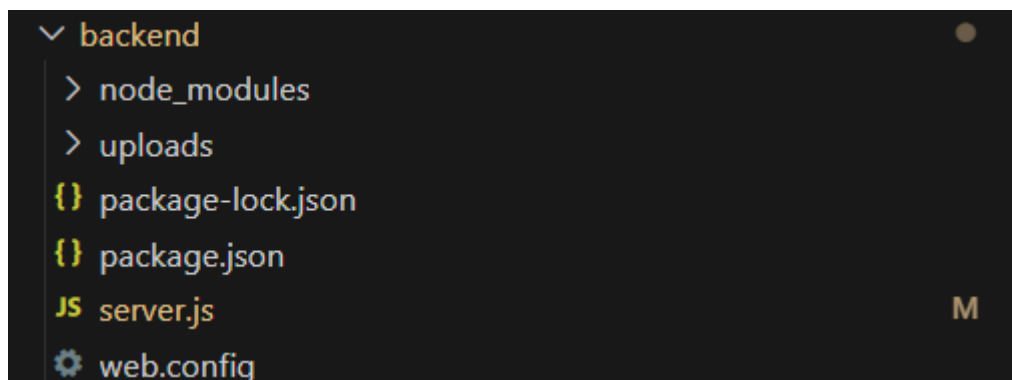


Figura 4. Estructura del Back-end

Para entender el funcionamiento de la aplicación, se van a detallar las rutas que se han empleado en el back-end.

9.2.1. Rutas relacionadas con los usuarios

POST /api/register: Esta ruta se encarga del registro de un nuevo usuario en la aplicación. Realiza la comprobación de que las contraseñas coincidan y después les aplica un hash. Si el correo proporcionado no existe en la base de datos, registra al nuevo usuario en la aplicación. Además en función del rol que haya seleccionado el usuario, realizará un insert en Cliente, Charanga o Musico. Si el registro ha sido posible, se enviará un correo al email indicado, con un mensaje de bienvenida indicando que el registro ha sido un éxito. Finalmente devuelve el token del usuario.

POST /api/login: Esta ruta se encarga del proceso de login de un usuario. Comprueba si la contraseña y el email introducidos sean correctos y estén en la base de datos. Si es correcto devuelve el token del usuario.

POST /api/perfil: Ruta que realiza la actualización del perfil de un usuario cuando elige la opción de Editar perfil. Actualiza los campos que hayan sido alterados, biografía, contraseña, foto de perfil o teléfono.

GET /api/usuario/:id/foto: Ruta que se utiliza para cargar la foto de perfil del usuario, que está almacenada en formato BLOB en la base de datos.

9.2.2. Rutas de utilidad

Estas rutas se utilizan en casi todas las páginas de la aplicación, ya que sirven para implementar funcionalidades comunes, relativas a instrumentos o provincias.

GET /api/instrumentos: Ruta que se utiliza para cargar los todos nombres de los instrumentos que existen. Se utiliza para el formulario de registro y para los filtros por instrumento.

GET /api/provincias: Ruta que se utiliza para cargar los todos nombres de las provincias que hay en España. Se utiliza también para el formulario de registro y para los filtros por provincia que hay en todas las páginas.

POST /api/setValoracionMedia: Esta ruta se utiliza cada vez que se guarda una valoración en la base de datos. El método busca todas las valoraciones que ha recibido un usuario, saca el valor medio y actualiza el campo valoracionMedia del usuario.

GET /api/valoracionMediaTipoActo: Esta ruta se encarga de recoger las valoraciones que ha recibido un usuario en función a un tipo de acto en concreto. Busca los valores en la base de datos, obtiene la media y lo devuelve. Esta ruta se utiliza a la hora de mostrar la valoración media en un tipo de acto determinado cuando se reciben solicitudes para las ofertas de charangas o para las solicitudes de los músicos a los actos.

GET /api/numero_usuarios: Esta ruta se utiliza en la página Home de la aplicación para mostrar la cantidad de cada rol de usuarios que hay registrados.

9.2.3. Rutas relacionadas con las charangas

GET /api/charangas: Esta ruta se emplea para cargar los datos de todas las charangas que hay registradas, de manera que aparecen en la página Nuestras charangas.

GET /api/anuncios_charangas: Esta ruta se emplea para cargar los anuncios de charangas que hay en la base de datos. Este método puede recibir parámetros como fecha de inicio, fecha de fin, provincia y en función de los parámetros que recibe devuelve unos anuncios u otros.

POST /api/crear_anuncio_charanga: Esta ruta se emplea para insertar un anuncio de charanga en la base de datos. Para ello primero realiza un insert en la tabla Anuncios y posteriormente otro en la tabla Anuncios_charangas, con el id del anuncio y el de la charanga.

9.2.4. Rutas relacionadas con los músicos

GET /api/anuncios_musicos: Esta ruta se emplea para cargar los anuncios de músicos que hay en la base de datos. Este método, al igual que el de anuncios charangas, puede recibir parámetros como fecha de inicio, fecha de fin, provincia, e Instrumento, y en función de los parámetros que recibe devuelve unos anuncios u otros.

POST /api/crear_anuncio_musico: Esta ruta se emplea para insertar un anuncio de músico en la base de datos. Para ello primero realiza un insert en la tabla Anuncios y posteriormente otro en la tabla Anuncios_músicos, con el id del anuncio y el del músico.

9.2.5. Rutas relacionadas con las ofertas de clientes para charangas

GET /api/ofertas: Ruta que se utiliza para cargar las ofertas que publican los clientes buscando charangas. Puede recibir parámetros como fecha inicio y fecha fin, tipo de acto y provincia de la oferta.

POST /api/crear_oferta: Esta ruta se emplea para insertar una nueva oferta en la base de datos. Recibe los parámetros de la oferta, como son el ID del cliente, título, descripción, dirección, tipo de acto, fecha inicio y fin, y la provincia donde se publica.

POST /api/crear_solicitud: Ruta que se utiliza para insertar una nueva solicitud en la base de datos. Cuando una charanga quiere solicitar participar en una oferta de un cliente, emplea este método para registrar su solicitud. El método recibe el ID de la oferta y el ID de la charanga. Una vez insertada la solicitud en la tabla solicitudes, envía un email al cliente informándole de que ha recibido una solicitud para esa oferta.

GET /api/solicitudes: Esta ruta se utiliza para obtener las solicitudes que ha recibido una oferta. De esta manera el cliente puede ver un listado de las solicitudes que ha recibido indicando los datos de la charanga, como su nombre, foto y valoración media.

GET /api/solicitudes_user: Esta ruta se utiliza para obtener las solicitudes que ha enviado una charanga. Gracias a esta ruta podemos cargar el estado de las solicitudes, y en base a ellas, mostrar en el Frontend si para una oferta aún no ha enviado solicitud, si está en estado enviada, aceptada o rechazada.

POST /api/aceptar_solicitud: Esta ruta recibe el id de la oferta y de la charanga que envió la solicitud. Con ellos actualiza el estado de la solicitud a Aceptada, añade el id de la charanga al campo idCharanga de la oferta y además rechaza todas las demás solicitudes que haya recibido la oferta. Una vez actualizadas todas las tablas, envía un email a la charanga indicando que su solicitud para esa oferta ha sido aceptada.

POST /api/rechazar_solicitud: Esta ruta recibe el id de la oferta y de la charanga que envió la solicitud, con ellos busca la solicitud que tenga idOferta e idCharanga iguales a los recibidos y cambia el estado de la solicitud de la charanga a Rechazada.

POST /api/asignar_valoracion_charanga: Esta ruta recibe el idOferta, idCharanga, la puntuación de la valoración y el tipo de acto de la oferta. Con estos datos, cambia el estado de la solicitud a Valorada, después guarda la valoración de la charanga en el campo valoracionCharanga de la oferta, y finalmente guarda la valoración en la tabla Valoraciones indicando el idUsuario igual a la charanga, y el tipo de acto en el campo tipoActo.

POST /api/asignar_valoracion_cliente: Esta ruta recibe el idOferta, idCharanga, la puntuación de la valoración y el id del cliente. Con estos datos, cambia el estado de la solicitud a Finalizada, después guarda la valoración del cliente en el campo valoracionCliente de la oferta, y finalmente guarda la valoración en la tabla Valoraciones indicando el idUsuario igual al del cliente, y en tipoActo no guarda nada, ya que de los clientes no nos interesa guardar este dato.

9.2.6. Rutas relacionadas con los actos de charangas para músicos

POST /api/crear_acto: Esta ruta se emplea para insertar un nuevo acto en la base de datos. Recibe los parámetros del acto, como son el ID de la charanga, título, tipo de acto, descripción, fecha inicio, fecha fin, ID de la provincia y un array con los músicos solicitados, en el que se indica el ID del instrumento y el número de músicos solicitados. Primero inserta el acto en la tabla actos, y después en función del número de músicos que haya recibido va realizando inserts en la tabla acto_instrumentos, donde guarda el ID del acto, el ID del instrumento buscado y la cantidad de músicos de ese instrumento.

GET /api/actos: Ruta que se utiliza para cargar los actos que publican las charangas buscando músicos. Puede recibir parámetros como fecha inicio y fecha fin, tipo de acto, instrumento buscado o provincia del acto.

POST /api/crear_solicitud_musico: Ruta que se utiliza para insertar una nueva solicitud en la base de datos. Cuando un músico quiere solicitar participar en un acto de una charanga, emplea este método para registrar su solicitud. El método recibe el ID de la oferta y el ID del músico. Una vez insertada la solicitud en solicitudes_musicos, se envía un email a la charanga informándole de que ha recibido una solicitud para ese acto.

GET /api/solicitudes_musico: Ruta que se utiliza para cargar las solicitudes que ha enviado un músico. Gracias a esta ruta podemos cargar el estado de las solicitudes, y en base a ellas, mostrar en el Frontend si para un acto aún no ha enviado solicitud, si está en estado enviada, aceptada o rechazada.

GET /api/solicitudes_acto: Ruta que se utiliza para cargar las solicitudes que ha recibido un acto publicado por una charanga. En base al ID del acto se listan las solicitudes que se han recibido.

POST /api/aceptar_solicitud_musico: Esta ruta recibe el id del acto, el ID del músico que envió la solicitud y su ID de instrumento. Con ellos actualiza el estado de la solicitud a Aceptada, después resta 1 a la cantidad de la tabla acto_instrumentos que tenga idActo e idInstrumento iguales al recibido, si al restar 1 la cantidad es 0, realiza un delete. Después resta 1 al campo músicosBuscados del acto que se está actualizando en la tabla actos y por último realiza un insert en la tabla musicos_contratados donde se guarda el ID del acto y el ID del músico. Una vez actualizadas todas las tablas, envía un email al músico indicando que su solicitud para ese acto ha sido aceptada.

POST /api/rechazar_solicitud_musico: Esta ruta recibe el ID del acto y del músico que envió la solicitud, con ellos busca la solicitud que tenga idActo e idMusico iguales a los recibidos y cambia el estado de la solicitud del musico a Rechazada.

GET /api/datos_musicos: Ruta que se utiliza para mostrar los datos de los músicos que han sido contratados para un acto, el método recibe un array de IDs de músicos y un ID de acto, y con ellos busca los datos de los músicos contratados.

GET /api/datos_musico: Ruta que se utiliza para mostrar la valoración que ha realizado un músico a una charanga en un acto en el que ha sido contratado. Recibe el ID del acto y el ID del músico.

POST /api/asignar_valoracion_charanga_acto: Esta ruta recibe el idActo, idCharanga, la puntuación de la valoración, el idMusico y el tipo de acto del acto. Con estos datos, cambia el estado de la solicitud a Valorada, después guarda la valoración de la charanga en el campo valoracionCharanga de musicos_contratados, y finalmente guarda la valoración en la tabla Valoraciones indicando el idUsuario igual a la charanga, y el tipo de acto en el campo tipoActo.

POST /api/asignar_valoracion_musico_acto: Esta ruta recibe el idActo, la puntuación de la valoración, el idMusico y el tipo de acto del acto. Guarda la valoración del músico en el campo valoracionMusico de musicos_contratados, y finalmente guarda la valoración en la tabla Valoraciones indicando el idUsuario igual al músico, y el tipo de acto en el campo tipoActo.

9.2.7. Autenticación y seguridad de la aplicación

Para llevar a cabo el inicio de sesión en la aplicación, se ha empleado un login a través de email y contraseña. Para guardar la contraseña en la base de datos primero usamos la librería bcrypt (Izertis, s.f.) para encriptar la contraseña.

Bcrypt es una función de hashing de passwords basado en el cifrado de Blowfish. Lleva incorporado un valor llamado salt, que es un fragmento aleatorio que se usará para generar el hash asociado a la password, y se guardará junto con ella en la base de datos. Así se evita que dos passwords iguales generen el mismo hash y los problemas que ello conlleva, por ejemplo, ataque por fuerza bruta a todas las passwords del sistema a la vez. Con el salt, se añade un grado de complejidad que evita que el hash asociado a una password sea único.

Bcrypt nos permite elegir el valor de saltRounds, que nos da el control sobre el coste de procesamiento de los datos. Cuanto más alto es este número, más tiempo requiere la máquina para calcular el hash asociado a la password, en nuestro caso, utilizaremos un valor de 10.

Una vez comprobado que la contraseña y el email son correctos, utilizamos la librería jsonwebtoken (OpenWebinars, s.f.) para el intercambio de información segura entre el Frontend y el Backend.

JWT (JSON Web Token) es un estándar que está dentro del documento RFC 7519.

En el mismo se define un mecanismo para poder propagar entre dos partes, y de forma segura, la identidad de un determinado usuario, además con una serie de privilegios.

Estos privilegios están codificados en objetos de tipo JSON, que se incrustan dentro de del cuerpo de un mensaje que va firmado digitalmente.

Gracias a jsonwebtoken, generamos un token para el usuario y devolvemos al frontend el token y la información del usuario en formato json.

Por otro lado el proceso de registro funciona de manera muy similar, si el email introducido por el usuario no existe ya en la base de datos y los datos introducidos en el

formulario son correctos, encripta la contraseña con bcrypt, genera el token para el usuario y lo devuelve al frontend en formato json.

En cuanto a la seguridad, hemos utilizado un certificado autofirmado para poder trabajar con HTTPS.

9.3. Front-end de la aplicación (Angular)

La parte principal de la aplicación ha sido el desarrollo del frontend, es la que más tiempo de desarrollo ha llevado.

El desarrollo ha sido mediante Angular, primero creamos un proyecto angular vacío y después fui generando los componentes con el comando:

```
ng generate component components/nombreComponente
```

y el servicio para autenticación con el comando:

```
ng generate service services/auth
```

Estos comandos realizan automáticamente los imports necesarios para el archivo app.module.ts. Después configuramos el archivo app-routing.module.ts para añadir las rutas de los componentes creados y comenzamos a desarrollar las diferentes vistas.

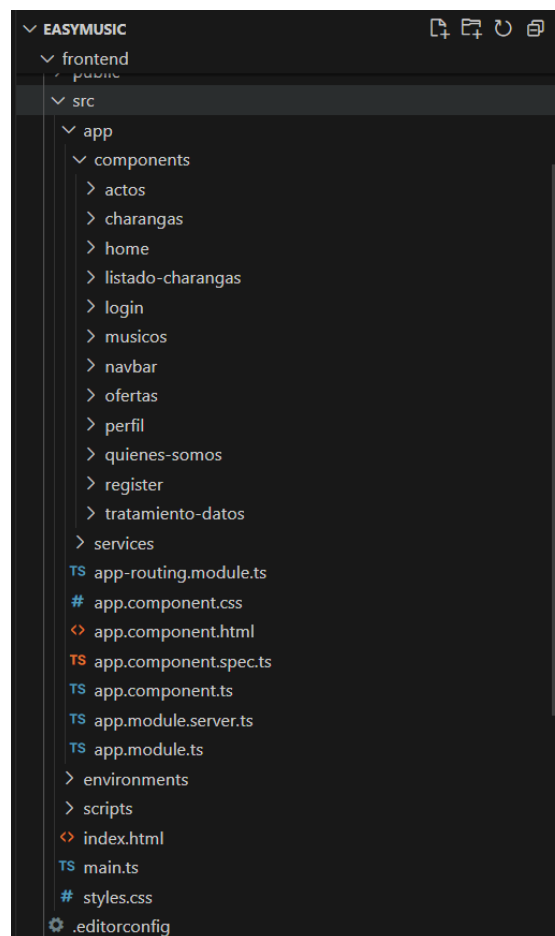


Figura 5. Estructura del frontend del proyecto

Como podemos ver en la imagen, el código de cada componente está en la ruta `src/app/components/suNombreComponente`. Por otro lado el código del servicio está en la ruta `src/app/services/authService`, y los ficheros principales en `src/app`.

Una vez entendido como se organizan los ficheros del frontend, se procede a comentar las vistas que se han desarrollado.

Las capturas de pantalla de las vistas del Frontend han sido tomadas desde un OnePlus 6T.

9.3.1. Barra de navegación

La barra de navegación es la parte con la que nos vamos a desplazar por las distintas pestañas de la aplicación. En ella podemos ver el logo de la aplicación, su nombre y un menú desplegable. El contenido del menú va cambiando en función de si hemos iniciado sesión o no, y en caso de haber iniciado sesión, se nos mostrarán unas funcionalidades u otras dependiendo del rol de usuario que tengamos.

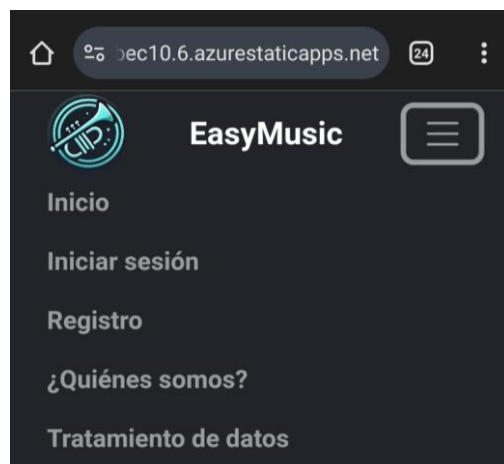


Figura 6. Barra de navegación para usuario sin iniciar sesión.

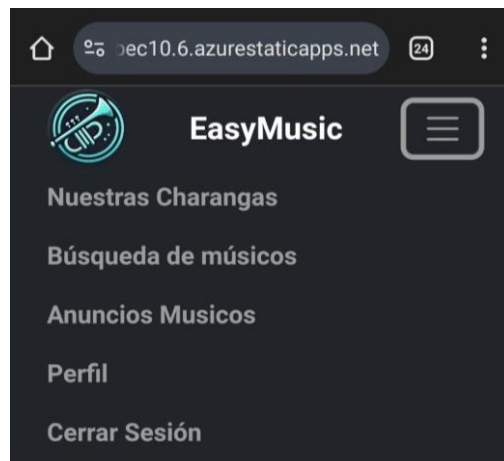


Figura 7. Barra de navegación para usuario músico



Figura 8. Barra de navegación para usuario charanga

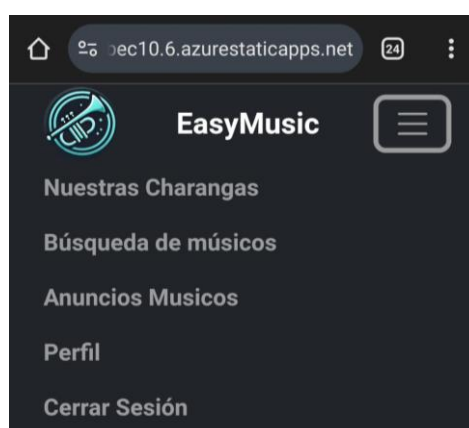


Figura 9. Barra de navegación para usuario cliente

9.3.2. Vista inicio

Al cargar la página se nos muestra la página de inicio. En ella podemos ver la imagen de la aplicación junto a su lema. Más abajo hay un mensaje de bienvenida con un pequeño texto que informa sobre la aplicación. Más abajo hay una sección que muestra el número de usuarios registrados con los que cuenta la aplicación. Finalmente hay un botón que nos dirige a la pestaña de Login.

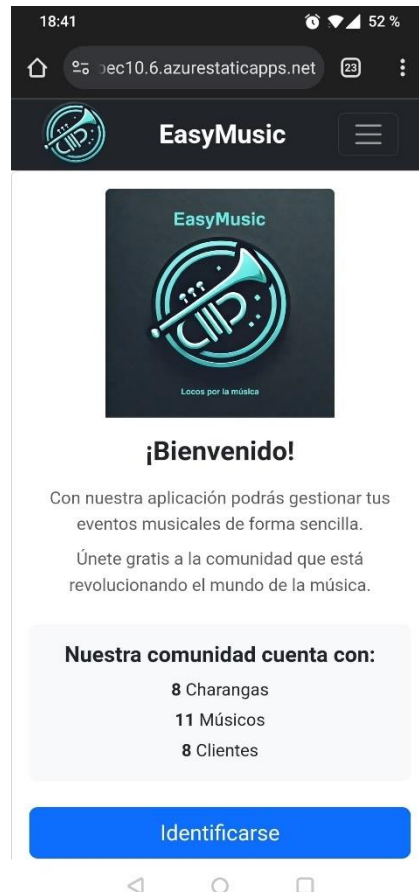


Figura 10. Vista inicio

9.3.3. Vista Iniciar sesión

En esta pestaña podemos ver nuevamente el logo de la aplicación y debajo de este un formulario para iniciar sesión. En él se nos pide el correo electrónico con el que nos registramos en la aplicación y la contraseña. Incluye un botón de un ojo para poder ver la contraseña introducida. En caso de no tener cuenta si pinchamos en Regístrate nos redirigirá a la sección Registro.

Cuando el usuario pulse en el botón de Ingresar, se lanzará una petición a la ruta del backend, y si la respuesta es correcta, el usuario ingresará en la aplicación.

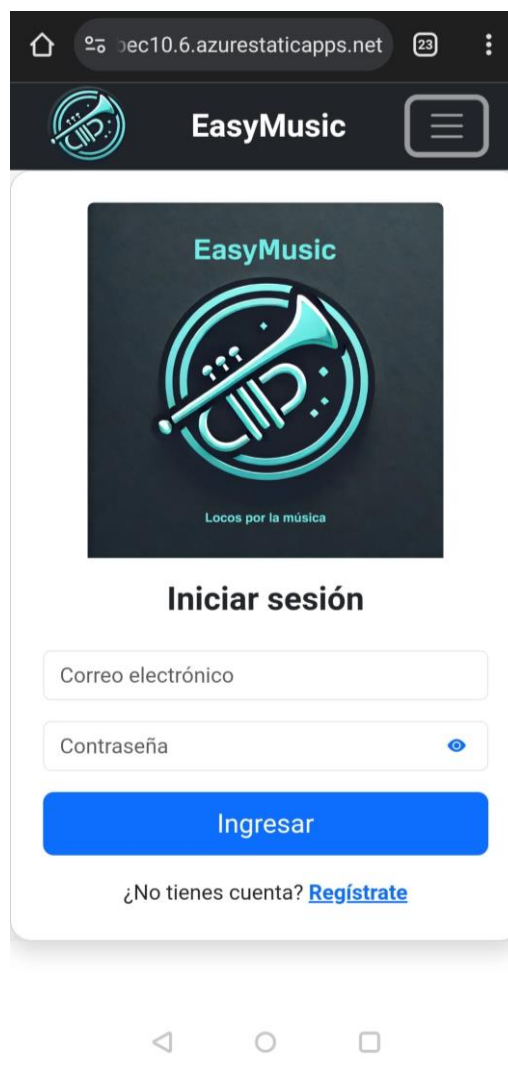


Figura 11. Vista iniciar sesión

9.3.4. Vista Registro

En esta página está compuesta por un formulario para crear el usuario. El formulario tiene una parte común a cualquier tipo de usuario y una parte que en función del rol que elija el usuario cambia. En cualquier caso, todos los campos son obligatorios y hasta que no sean rellenados no se podrá lanzar el registro. La parte común está compuesta por:

- Foto de perfil: El usuario puede seleccionar una imagen de la galería o tomar una con la cámara. La foto de perfil que seleccione será la que aparecerá en los anuncios, actos u ofertas.
- Nombre: Será el nombre con el que se llamará al usuario.
- Email: El email será importante, ya que cuando quiera anunciarse, se utilizará para que los demás usuarios puedan contactar con él y además recibirá en él las notificaciones cuando reciba una solicitud o cuando una solicitud sea aceptada.
- Contraseña y confirmar contraseña: Estos campos serán la contraseña para iniciar sesión del usuario, se les aplicará un hash con bcrypt en el backend a la hora de guardarlas en base de datos.
- Biografía: Campo descriptivo sobre el usuario. En el caso de las charangas será mostrado en la pestaña Nuestras charangas para conocer un poco más sobre ellas.
- Teléfono: Otro campo muy importante, será utilizado para proporcionar los datos de contacto del usuario cuando se promocióne o cuando sea contratado en una oferta o acto.
- Provincia. Este campo se utilizará cuando se apliquen filtros por provincia, para encontrar charangas o músicos de una determinada provincia.
- Rol: Con esta selección elegiremos si queremos ser músico, cliente o charanga. Según el rol que elijamos a continuación aparecerán más campos.

Si elegimos rol cliente no se añadirá ningún campo más y podremos realizar el registro.

En caso de seleccionar rol Charanga, aparecerá lo siguiente:

- Año de fundación: Año en el que fue fundada la charanga, para conocer cuanto tiempo llevan en activo.

Finalmente, si seleccionamos rol Músico, aparecerán los siguientes campos:

- Instrumento: Aquí el músico podrá seleccionar el instrumento que toca, de cara a solicitar participar en actos de charanga o para anunciarse.

- Estudios musicales: El músico indicará su nivel de estudios para que la charanga sepa con mayor exactitud el nivel del músico.
- Tienes coche?: Este campo es una casilla que se puede marcar o dejar desmarcada. A la hora de ser contratado por una charanga, el coche es un apartado importante, ya que para planificar como acude el músico al acto es un factor para considerar.

Ahora vamos a ver las vistas del formulario de registro para el caso de cliente:

The screenshot shows a mobile web browser with the URL 'ec10.6.azurestaticapps.net'. The app header is 'EasyMusic' with a menu icon. The page title is 'Registro'. Below the title is a 'Foto de perfil *' field with a blue circular placeholder. Underneath are two buttons: 'Seleccionar archivo' and 'Ningún arc...leccionado'. The form continues with 'Nombre *', 'Email *', 'Contraseña *', and 'Confirmar Contraseña *' fields, each with a corresponding input box and a toggle eye icon. The 'Biografía *' field is a text area. At the bottom are standard Android navigation icons.

Figura 12. Vista registro parte 1

This screenshot shows the continuation of the registration form. It starts with the 'Biografía *' field, which is a text area containing the placeholder 'Describe algo sobre ti...'. Below it is the 'Teléfono *' field with a placeholder 'Ej: 666555444'. The 'Provincia *' field is a dropdown menu with 'Selecciona una provincia' and a red error message 'Selecciona una provincia.' below it. The 'Rol *' field is a dropdown menu with 'Cliente' selected. At the bottom is a blue 'Registrarse' button. Standard Android navigation icons are at the very bottom.

Figura 13. Vista registro parte 2

9.3.5. Vista ¿Quiénes somos?

En esta pestaña es de carácter informativo, en ella se muestra un texto sobre mi persona, donde explico el porqué de hacer este proyecto, los objetivos del proyecto y un breve resumen sobre mi vida musical.

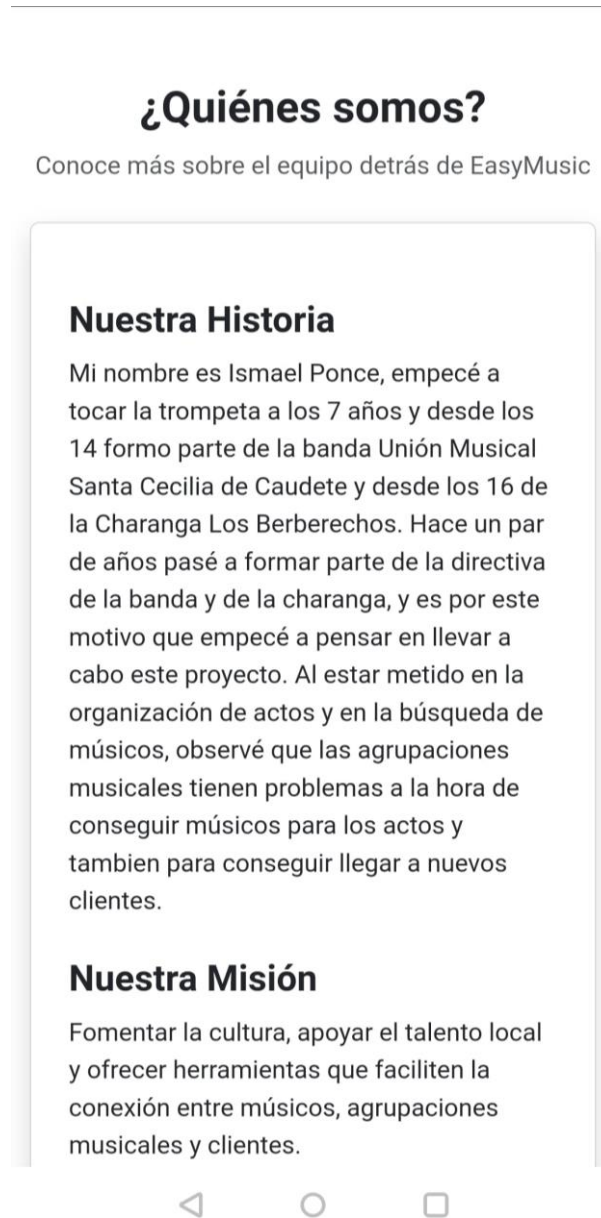


Figura 14. Vista ¿Quiénes somos?

9.3.6. Vista Tratamiento de datos

En esta pestaña es meramente informativa, en ella se describe el fin con el que la aplicación recoge datos. Se puede acceder a ella desde el botón de más información en el aviso de Cookies o desde la barra de navegación.

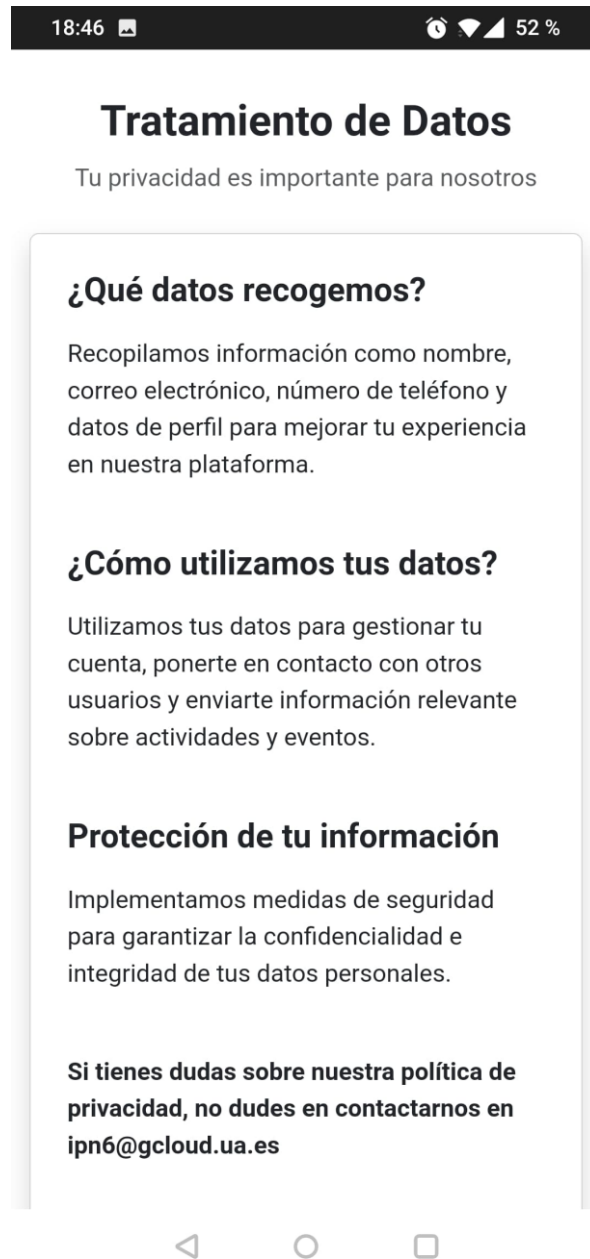


Figura 15. Vista Tratamiento de datos

9.3.7. Vista Perfil

Pestaña que se abre de manera predeterminada cuando iniciamos sesión. En ella se detallan los datos del usuario, como foto de perfil, nombre, email, teléfono, provincia, rol, biografía y su valoración media. En caso de ser músico se mostrará su instrumento, nivel musical y si tiene coche. Y en caso de ser charanga se mostrará su año de fundación.

Al final se muestra un botón para Editar el perfil, en caso de pulsarlo se abrirá una ventana en la que podremos editar nuestro perfil. Podremos cambiar la foto de perfil, nombre, teléfono, biografía y contraseña. Al pulsar el botón, el método del backend actualizará los cambios que hayan sido modificados.

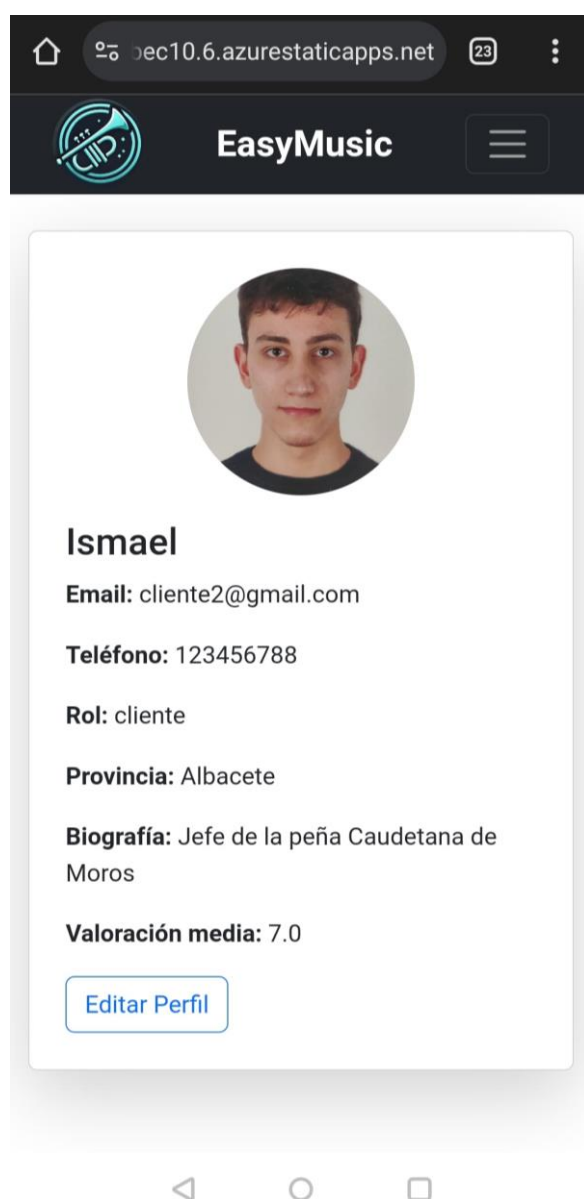


Figura 16. Vista perfil

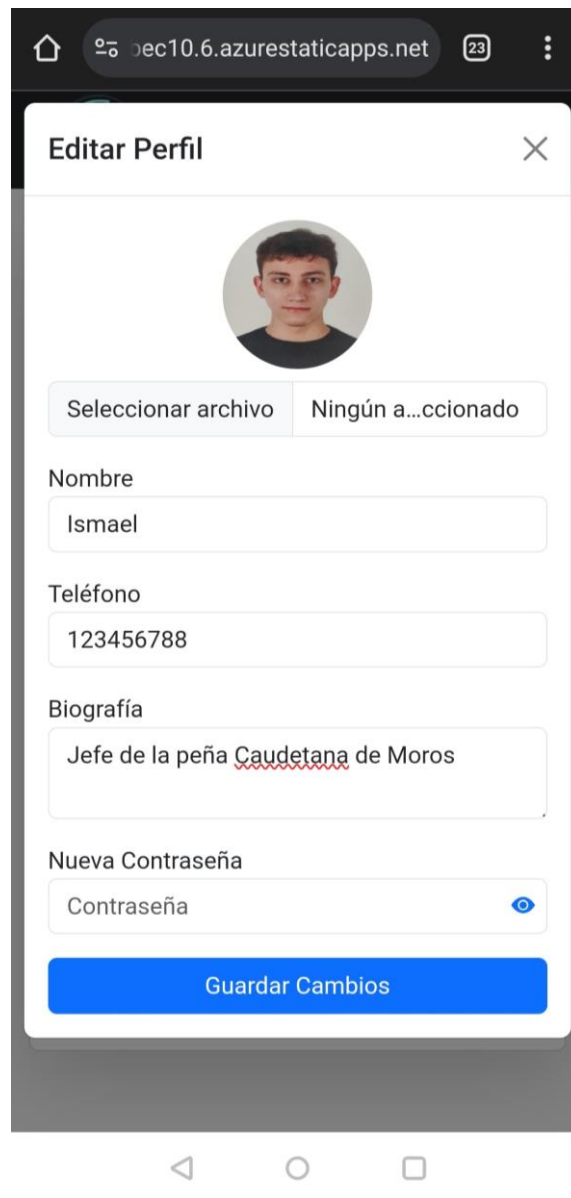


Figura 17. Vista editar perfil

9.3.8. Vista Nuestras Charangas

En esta pestaña se muestra un listado con todas las charangas que se han unido a la aplicación. Al inicio de la pestaña podemos ver un botón para filtrar charangas por la provincia seleccionada o simplemente verlas todas.

Para cada charanga se muestra una tarjeta con su foto de perfil, su nombre, correo, teléfono, la provincia a la que pertenece, su año de fundación, la biografía y su valoración media, lo que nos permitirá identificar las mejores charangas.



Figura 18. Vista Listado Charangas

9.3.9. Vista Ofertas para Charangas

Esta es la pestaña más atractiva para el cliente. En ella el cliente puede crear ofertas solicitando charangas. Al inicio de la página hay un botón para publicar nuevas ofertas, al pinchar en él se abrirá un formulario para crear una nueva oferta. En él, el cliente debe introducir los siguientes datos:

- Título: El título que identificará la oferta para las charangas.
- Tipo de acto: Este campo es de tipo desplegable, en él el cliente puede seleccionar entre distintos tipos de acto. Este campo servirá para poder identificar en qué consistirá el acto, las opciones se han dividido en: Desfile, pasacalles, concierto, Día completo, Cumpleaños o serenata y por último boda, bautizo o comunión. De esta manera las charangas posteriormente podrán filtrar las ofertas en función de su tipo.
- Descripción: Campo para introducir un resumen informativo sobre lo que consistirá el acto.
- Fecha inicio: Día en el que comenzará la actuación.
- Fecha fin: Día en el que comenzará la actuación. Si la oferta se lleva a cabo en un mismo día deberá ser igual a la fecha de inicio.
- Provincia: Campo en el que el cliente tiene que introducir la provincia donde tendrá lugar el acto. Servirá para que las charangas busquen ofertas que sean cercanas.
- Dirección: Campo donde el cliente debe detallar la dirección donde tendrá lugar el acto, como localidad y calle.

Una vez introducidos todos los campos el cliente pulsará en el botón “Publicar” y la oferta se registrará en la aplicación. Una vez publicada, las charangas podrán enviar una solicitud para participar en ella.

En la página se pueden filtrar las ofertas en función de fecha inicio y fin, provincia y tipo de acto.

Para cada oferta se mostrará una tarjeta, la tarjeta está dividida en dos secciones. En la superior se muestran los datos del cliente: Nombre, valoración media y foto de perfil. En la sección inferior se muestran los datos de la oferta: Título, tipo de acto, descripción,

provincia, fecha, dirección y el estado de la oferta, que puede ser: Sin contratar, contratada, valorada y finalizada.

Cuando los clientes reciben una solicitud, en su correo se mostrará un email indicando que han recibido una solicitud para una determinada oferta. Cuando el cliente abra la aplicación, seleccionará el botón “Ver detalles” de la oferta y podrá ver un listado con las solicitudes que ha recibido, donde se muestran los detalles de la charanga. Los detalles más importantes de las charangas son su Valoración media general y su Valoración media para ese tipo de acto. El cliente tendrá un botón para aceptar esa solicitud o para rechazarla. Si acepta la solicitud, la charanga recibirá un correo indicando que su solicitud ha sido aceptada. Una vez aceptada la solicitud, se mostrará en la ventana “Ver detalles” los datos de email y teléfono de la charanga y del cliente para que se puedan poner en contacto.

Una vez finalizado el acto, el cliente asignará una valoración a la charanga y posteriormente la charanga asignará una valoración al cliente, las cuales servirán para actualizar las valoraciones medias de los usuarios.

Esta pestaña actúa de forma diferente en función de si la ve un cliente o una charanga. El cliente dispone del botón “Crear Oferta” y cuando publica una oferta se muestra siempre el botón “Ver detalles”. Mientras que si accedemos desde una charanga, solo podemos ver el listado de ofertas e inicialmente nos aparecerá para cada oferta un botón “Me interesa!”. Al pulsar en el botón, el botón cambiará a “Solicitado” y aparecerá desactivado en gris. Si la solicitud es rechazada cambiará a “Rechazada” en color rojo, mientras que si es aceptada aparecerá el botón para ver detalles.



EasyMusic

Mis ofertas

Crear oferta

Cerrar

Título

Tipo de acto

Descripción

09/05/2025

09/05/2025

Selecciona una provincia

Dirección

Publicar

Figura 19. Vista Ofertas para charangas, sección Crear oferta.

18:48 52 %

Publicar nueva oferta

Filtrar ofertas

Todas las provincias

09/05/2025

09/06/2025

Todos los tipos de acto

Ismael

Valoración media: 7.0



Fiestas de San Isidro

Tipo de acto: Pasacalles

Descripción: Buscamos Charanga para un pasacalles en madrid

Fecha: 18/05/2025 - 18/05/2025

Provincia: Madrid

Dirección: Calle alicante 24

Estado: Finalizada

Ver detalles

Figura 20. Vista Ofertas para charangas, sección listado de ofertas del cliente

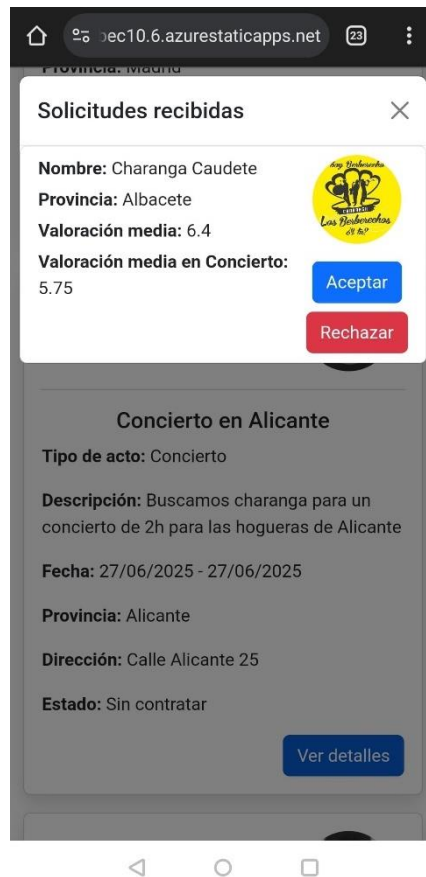


Figura 21. Vista Ofertas para charangas, sección solicitudes recibidas



Figura 22. Vista Ofertas para charangas, sección solicitud aceptada

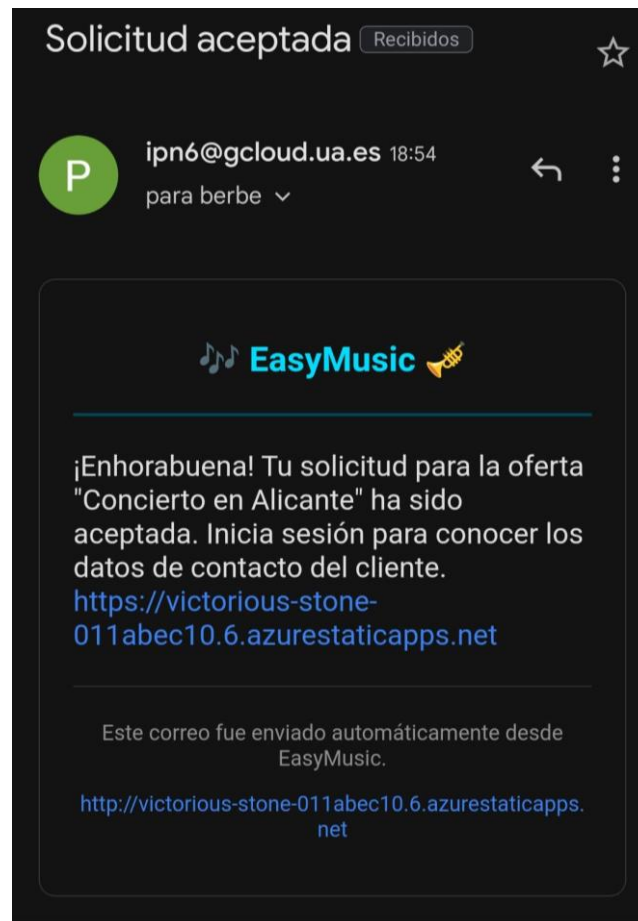


Figura 23. Vista Ofertas para charangas, ejemplo de correo.

9.3.10. Vista Búsqueda de músicos

En esta vista permite completar sus plantillas a las charangas. Al inicio hay un botón que permite abrir un formulario para crear actos. En este formulario la charanga tiene que introducir:

- Título: Nombre que identificará al acto.
- Tipo de acto: Al igual que en ofertas, la charanga especificará el tipo de acto de un seleccionable.
- Descripción: La charanga especificará una descripción para el acto.
- Fecha de inicio: Fecha de inicio del acto.
- Fecha de fin: Fecha de fin del acto.
- Provincia: Provincia en la que se desarrollará el acto.
- Número e instrumentos buscados: En esta sección, la charanga introducirá los músicos que necesita para el acto. En la casilla de la izquierda especificará el número y en el desplegable seleccionará el instrumento. Si desea más músicos de otro instrumento pulsará en el botón “+ Añadir más músicos”, y se abrirá una nueva línea para introducir músicos para otro instrumento.

Una vez rellenados los datos, la charanga pulsará en el botón “Publicar” y el acto se añadirá a la base datos.

Debajo del botón publicar nuevo acto, hay una sección para filtrar actos, donde se pueden filtrar por provincia, fecha inicio y fecha fin y por instrumentos buscados.

Para cada acto se mostrará una tarjeta en la que se mostrarán dos secciones: En la parte superior se mostrarán los datos de la charanga: Nombre, valoración media y foto de perfil. En la sección inferior se mostrará: Nombre del acto, tipo de acto, descripción, fecha de inicio y de fin, y la provincia.

Si el usuario es charanga, se mostrará un botón “Ver detalles”, que al pulsar se abrirá una ventana emergente donde aparecerán las solicitudes de músicos que ha recibido para ese acto. Para cada solicitud se mostrarán los datos de cada músico, los cuales son: Nombre, provincia, instrumento, foto de perfil, estudios musicales, si tiene coche, valoración media y valoración media para ese tipo de acto.

Si una solicitud de un músico es aceptada, se mostrará en la pestaña en una nueva sección de músicos contratados, donde se especifican los datos de contacto del músico como teléfono y email. Además aparecerá un botón para valorar la actuación del músico, cuya valoración servirá para actualizar la valoración media del músico.

Desde la vista del músico, cuando vea el listado de actos se mostrará un botón de “Me interesa!” para cada acto. Cuando envíe una solicitud el botón pasará a estado desactivado y pondrá “Solicitado” y se enviará un correo avisando a la charanga de que ha recibido una solicitud, si la solicitud es rechazada el botón pasará a estado desactivado y pondrá “Rechazada”, y si la solicitud se acepta, se mostrará para el músico “Ver detalles”, donde al pulsar aparecerá una ventana emergente donde se mostrarán los datos de la charanga junto a sus datos de contacto, como son el email y el teléfono. Además se mostrará la sección para valorar a la charanga.

mis10.6.azurestaticapps.net 23

EasyMusic

Mis búsquedas de músicos

Información del acto Cerrar

Título

Tipo de acto

Descripción

09/05/2025

09/05/2025

Selecciona una provincia

Número e instrumentos buscados

1 Trombón

+ Añadir más músicos

Publicar

Figura 24. Vista búsqueda músicos, crear acto.

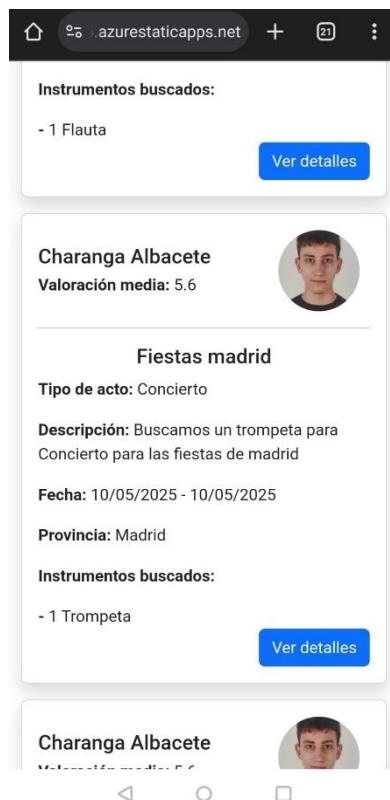


Figura 25. Vista búsqueda músicos, listado actos.



Figura 26. Vista búsqueda músicos, solicitudes recibidas.

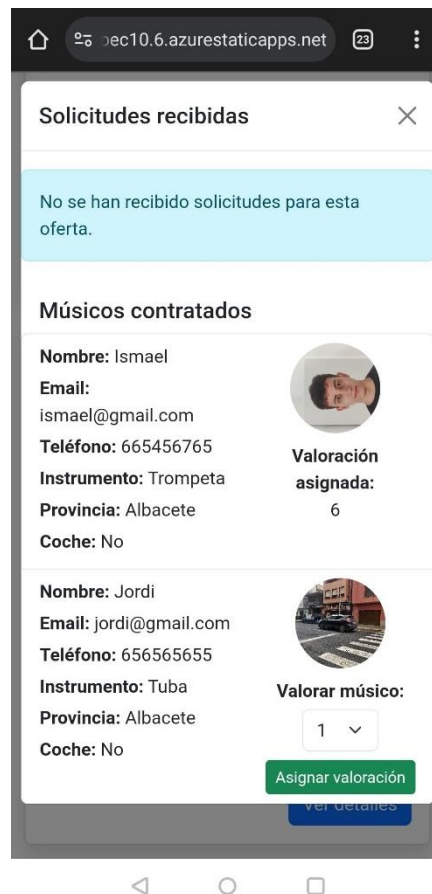


Figura 27. Búsqueda músicos, músicos contratados.

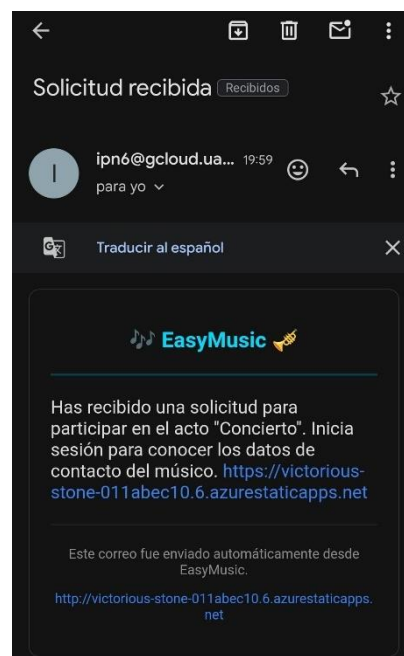


Figura 28. Búsqueda músicos, ejemplo correo.

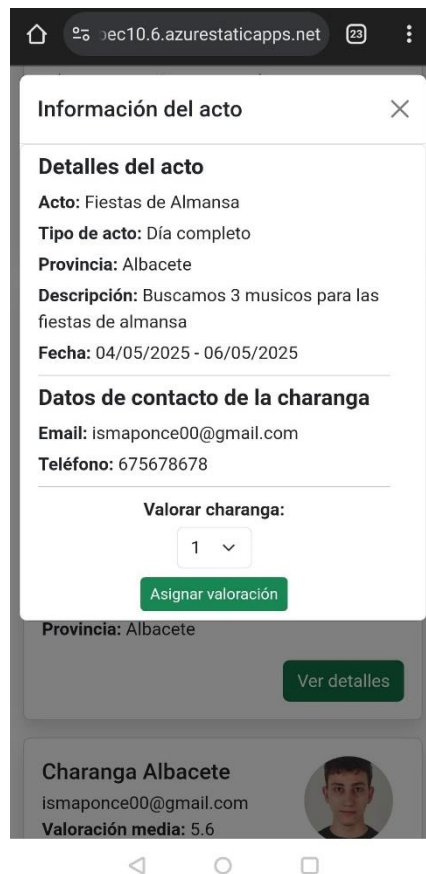


Figura 29. Búsqueda músicos, vista del músico.

9.3.11. Vista Anuncios Charangas

En esta pestaña las charangas pueden publicar anuncios de cara a que los clientes sepan que están disponibles. Al abrir la pestaña, las charangas disponen de un botón para crear anuncios. Al pulsar en él, se abre un formulario con los siguientes campos:

- Título: Nombre descriptivo para el anuncio.
- Descripción: Campo para introducir información sobre el anuncio.
- Fecha inicio: Fecha de inicio del anuncio.
- Fecha fin: Fecha de fin para el anuncio.
- Provincia: La charanga podrá seleccionar la provincia donde promocionarse mediante un desplegable.

Más abajo hay una sección de filtros para los anuncios. Los anuncios podrán ser filtrados por provincia, fecha inicio y fecha fin.

Finalmente se muestra el listado de anuncios de las charangas. Para cada anuncio se muestra una tarjeta, que está dividida en dos secciones. En la sección superior aparece el nombre de la charanga, su valoración media y su foto de perfil. En la sección inferior aparecen los datos del anuncio, como son el título, email de la charanga, teléfono de la charanga, provincia, descripción y fechas de inicio y fin del anuncio.

The screenshot shows a mobile application interface for 'EasyMusic'. At the top, there's a dark header with the app's logo and name. Below this, the section 'Anuncios de Charangas' is highlighted. A modal form titled 'Crear anuncio' is open, featuring a 'Cerrar' button in the top right corner. The form contains five input fields: 'Título', 'Descripción', '11/05/2025' (for start date), '11/05/2025' (for end date), and a dropdown menu labeled 'Selecciona una provincia'. A green 'Publicar' button is located at the bottom right of the form. Below the form, there is a section titled 'Filtrar anuncios'.

Figura 30. Anuncios charangas, crear anuncio.



Figura 31. Anuncios charangas, listado anuncios.

9.3.12. Vista Anuncios Músicos

Esta pestaña es bastante similar a la de anuncios de charangas, sin embargo aquí los músicos publican sus anuncios para que las charangas puedan consultar su disponibilidad.

Al inicio de la página aparece un botón para crear anuncio para los músicos, al pulsar en él se abre un formulario donde el músico tendrá que introducir:

- Título: Nombre descriptivo para el anuncio.
- Descripción: Campo para introducir información sobre el anuncio.
- Fecha inicio: Fecha de inicio del anuncio.
- Fecha fin: Fecha de fin para el anuncio.
- Provincia: El músico podrá seleccionar la provincia donde promocionarse.

Al pulsar en el botón publicar, el anuncio será introducido en la base de datos mediante la ruta del backend.

Más abajo hay una sección de filtros para los anuncios. Los anuncios podrán ser filtrados por provincia, instrumento o fecha inicio y fecha fin.

Por último se muestra el listado de anuncios de los músicos. Para cada anuncio se muestra una tarjeta, que está dividida en dos secciones. En la sección superior aparece el nombre del músico, su valoración media y su foto de perfil. En la sección inferior aparecen los siguientes datos:

- Título del anuncio.
- Email del músico con el que podrá recibir ofertas.
- Teléfono del músico con el que podrá ser contactado.
- Estudios musicales: Nivel musical que servirá para que las charangas aproximen de mejor manera su nivel.
- Provincia: Provincia donde se oferta el músico.
- Descripción: Información adicional para el anuncio, de manera que pueda resultar más atractivo.
- Coche: Indica si el músico dispone de coche para desplazarse al acto.
- Fechas: Fechas en las que está disponible el músico.



EasyMusic

Anuncios de Músicos

Crear anuncio Cerrar

Título

Descripción

09/05/2025

09/05/2025

Selecciona una provincia

Publicar

Filtrar anuncios

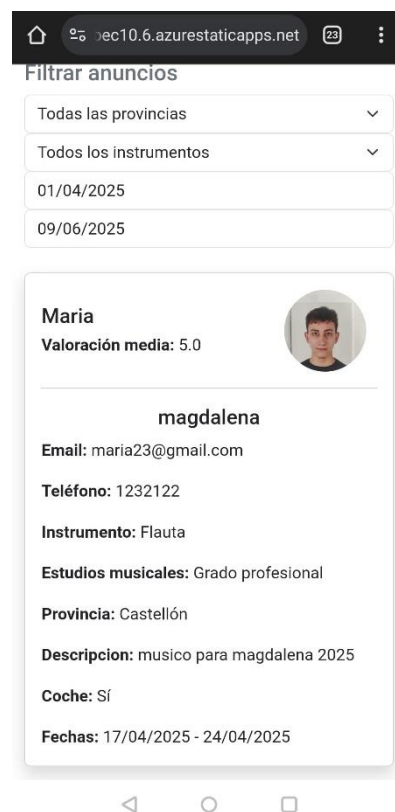
Todas las provincias

Todos los instrumentos

01/04/2025

09/06/2025

Figura 32. Anuncios músicos, crear anuncio.



Filtrar anuncios

Todas las provincias

Todos los instrumentos

01/04/2025

09/06/2025

Maria

Valoración media: 5.0



magdalena

Email: maria23@gmail.com

Teléfono: 1232122

Instrumento: Flauta

Estudios musicales: Grado profesional

Provincia: Castellón

Descripción: musico para magdalena 2025

Coche: Sí

Fechas: 17/04/2025 - 24/04/2025

Figura 33. Anuncios músicos, listado anuncios.

9.4. Procedimientos con mayor complejidad

Después de detallar las vistas del Frontend, voy a señalar cuales han sido los procesos más complejos a la hora de programar la aplicación.

9.4.1. Vista ofertas para charangas

El estado de esta vista depende de varios factores. Primero tenemos que tener en cuenta si el usuario es cliente o charanga.

Si el usuario es cliente, recogeremos todas las ofertas que hay en la base de datos y después aplicaremos una función map para transformar el array de ofertas y quedarnos solo con las ofertas cuyo idCliente es igual al id del usuario que ha iniciado sesión. De esta manera mostraremos todas las ofertas que ha publicado ese cliente.

Después, si queremos ver los detalles de una oferta, el manejo de la vista depende del estado de la oferta. Para saber el estado de la oferta, en la base de datos guardamos un campo booleano llamado “contratada”, si este valor es igual a 0 significa que la oferta no ha sido contratada, mientras que si es igual a 1, significa que ha sido contratada. Cuando pulsamos en “Ver detalles”, llamamos a una ruta del backend que nos devuelve todas las solicitudes que ha obtenido esa oferta.

Las solicitudes tienen un campo “estado” que nos sirve para saber en qué situación se encuentran, ese campo puede admitir los siguientes valores:

- Pendiente: Cuando la charanga ha enviado una solicitud y aún no ha sido aceptada o rechazada por el cliente.
- Rechazada: Cuando el cliente rechaza la solicitud.
- Aceptada: Si el cliente acepta la solicitud.
- Valorada: Si el cliente ha valorado a la charanga.
- Finalizada: Si la charanga ha valorado al cliente.

Al pulsar en “Ver detalles”, llamamos a una función que si alguna solicitud está en estado “Aceptada”, la guarda en la variable solicitudAceptada, para ser mostrada posteriormente. En función del estado de la oferta hay dos vistas:

- Si la oferta no ha sido contratada aún, obtendremos todas las solicitudes cuyo idOferta sea igual al de esa oferta y las mostraremos, indicando los datos de cada charanga.
- Si la oferta ha sido contratada, entonces cargaremos los datos de la solicitudAceptada y mostraremos los datos de esa charanga. Además aparecerá el apartado para valorar la charanga, que dependerá del estado de la solicitudAceptada:
 - Si el estado de la solicitud es “Aceptada”, al cliente se le mostrará el botón para asignar la valoración de la charanga. Una vez pulse en valorar, la valoración se registrará y el estado de la solicitud cambiará a “Valorada”.
 - Si el estado de la solicitud es “Valorada o Finalizada”, al cliente se le mostrará la valoración que ha asignado a esa charanga.

Por otro lado, si el usuario es charanga, al cargar la página obtiene todas las ofertas en función de los filtros. Después obtiene todas las solicitudes que ha realizado y en función del estado de esas solicitudes pinta el botón de una manera u otra.

Si una oferta no tiene solicitud se mostrará un botón “Me interesa”, si tiene solicitud pendiente aparecerá “Pendiente”, si ha sido rechazada aparecerá “Rechazada” y si ha sido aceptada se mostrará “Ver detalles”.

Al pulsar en “Ver detalles” aparecerán los datos de contacto del cliente y en función del estado de la solicitud aparecerá una sección para valorar al cliente:

- Si el estado es “Valorada”, se mostrará el botón para valorar al cliente, que al pulsarlo el estado cambiará a “Finalizada”.
- Si es estado es “Finalizada”, se mostrará la valoración asignada al cliente.

9.4.2. Vista búsqueda de músicos

Para mostrar esta vista he tenido que emplear algunos procedimientos complejos.

En el formulario de crear acto, a la hora de guardar los músicos que solicita una charanga para un acto, creo un array de músicos que está compuesto por cantidad e id de instrumento. Cada fila de músico buscado que se añade se guarda en ese array. Después, cuando mandamos los datos del formulario, en el backend se leen los datos del array de músicos y para cada elemento del array, se hace un insert en la tabla `acto_instrumentos` con la cantidad de músicos y el id del instrumento. Y mediante un contador, se suma el número total de músicos buscados para luego almacenarlo en el campo `músicosBuscados` de la tabla `Acto`.

Por otro lado, la forma de mostrar el listado de actos es similar al de ofertas, el manejo depende del tipo de usuario.

Si el usuario es charanga, se toman todos los actos que coincidan con los filtros de la tabla `actos` y se realiza un map para quedarnos con los actos cuyo `idCharanga` es igual al del usuario. Al pulsar en el botón “Ver detalles”, se cargan todas las solicitudes que ha recibido ese acto y los datos de los músicos que han sido aceptados, que se mostrarán en el apartado `Musicos contratados`, donde se podrá asignar una valoración. Al cargar los datos de los músicos también cargamos la valoración de ese músico, si aún no ha sido asignada, se mostrará el botón para valorarlo, y si ya tiene valoración, se mostrará la valoración asignada.

Si el usuario es músico, se muestran todos los actos que coinciden con los filtros. Si los instrumentos que se buscan en un acto no coinciden con los de ese usuario, no se mostrará el botón “Me interesa”, mientras que si el instrumento buscado coincide con el suyo, sí que aparecerá el botón.

Para manejar el estado del botón, al cargar la página se obtienen todas las solicitudes de ese usuario, y al igual que en ofertas, el estado del botón depende de la solicitud. Su estado puede ser “Pendiente”, “Rechazada”, o “Aceptada”, que en este caso se mostrará el botón “Ver detalles”. Al pinchar en él se le mostrarán los datos de la charanga y se cargarán los datos del músico para ese acto, en los que se incluyen la valoración a la charanga. Si aun

no la ha valorado, se mostrará el botón para valorar a la charanga, y si ya ha sido valorada, se mostrará la valoración que le ha asignado.

9.5. Despliegue de la aplicación

Para el despliegue de la aplicación se ha realizado un proceso dividido en 3 partes.

9.5.1. Despliegue Base de datos con Servidor flexible for MySQL de Azure

Primero se ha desplegado la base de datos en un servidor flexible for MySQL de Azure. Dentro de la página de Azure desplegué un servidor en España, descargué un certificado SSL para permitir conexiones seguras con la base de datos. Después configuré una regla Allow All para que permita conexiones desde cualquier IP, que va desde la 0.0.0.0 hasta la 255.255.255.255.

Finalmente, Azure proporciona los detalles para configurar la conexión a la base de datos, como son el hostname, puerto, usuario, contraseña y que requiera el modo SSL. Estos parámetros son introducidos en el código del backend y al ejecutar el código la conexión a la base de datos es efectiva.

9.5.2. Despliegue Backend con App Service de Azure

En segundo lugar se realizó el despliegue del Backend. Para ello se creó un App Service en Azure. Para configurar el App Service, introduje una serie de parámetros para almacenar la conexión a la base de datos, de manera que no tenga que introducir las credenciales dentro del código del server.js.

DB_HOST	 Mostrar valor	App Service	
DB_NAME	 Mostrar valor	App Service	
DB_PASSWORD	 Mostrar valor	App Service	

Figura 34. Parámetros conexión con la Base de Datos

De esta manera, en el código cuando queramos hacer referencia a alguna variable de la conexión, emplearemos por ejemplo: `${DB_HOST}`.

Después de configurar las variables, vinculé el código de GitHub con el App Service. Azure dispone de una herramienta para realizarlo de manera muy sencilla, de manera que mediante el Centro de implementación, tuve que vincular mi cuenta de GitHub, elegir el proyecto deseado e indicar la rama de la que debe obtener el código, y eligiendo la pila del entorno en tiempo de ejecución, en nuestro caso Node versión 20, quedando de la siguiente manera:



Figura 35. Centro implementación App Service Azure

9.5.3. Despliegue Frontend con Aplicación Web Estática de Azure

En último lugar se llevó a cabo el despliegue del Frontend. Para ello se creó una Aplicación Web Estática en Azure. Durante la configuración de la creación de la aplicación web, solamente hubo que seleccionar que la implementación fuera con el proyecto de GitHub.

Por último solo quedaba subir una versión para producción para que la Aplicación Web Estática pueda implementar la aplicación en un dominio. Por tanto, en nuestro proyecto local, cada vez que realizamos una nueva versión lanzamos este comando desde el directorio /Frontend:

```
ng build --configuration production
```

Una vez lanzado el comando, se crea una versión para producción en el directorio /dist/Frontend y solo queda subir el proyecto a GitHub.

Para comprobar si el despliegue ha sido correcto, nos vamos a nuestro proyecto GitHub, y en el apartado Deployments, podemos ver que el despliegue para producción ha sido un éxito:

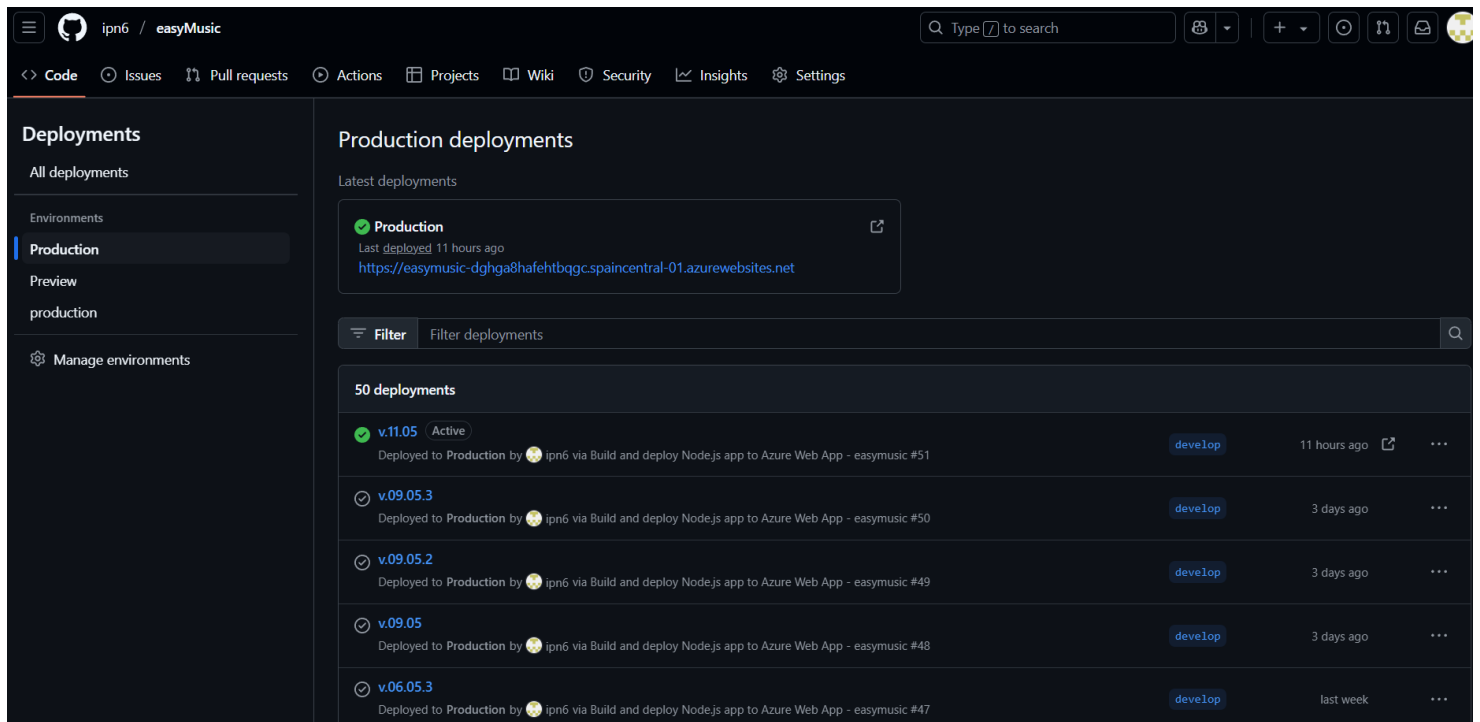


Figura 36. Despliegues para producción GitHub

9.5.4. Dominio de la aplicación

Una vez realizados todos los pasos, ya solo nos queda acceder a la Url desde nuestro dispositivo móvil y podemos disfrutar de la aplicación. En este caso nuestra aplicación ha sido desplegada en la siguiente Url:

<https://victorious-stone-011abec10.6.azurestaticapps.net>



Figura 37. Ejemplo aplicación corriendo

9.5.5. Usuarios para pruebas

Para facilitar la experiencia con la página, se han creado unos usuarios para que los evaluadores puedan acceder a ella desde los 3 tipos de usuarios distintos, y así probar todas las funcionalidades de la aplicación. Los usuarios facilitados han sido los siguientes:

- Rol cliente:
 - Usuario: cliente12345@gmail.com
 - Contraseña: Cliente12345
- Rol músico:
 - Usuario: musico12345@gmail.com
 - Contraseña: Musico12345
- Rol charanga:
 - Usuario: charanga12345@gmail.com
 - Contraseña: Charanga12345

10. Conclusiones

10.1. Resultados

El desarrollo de esta aplicación móvil para agrupaciones musicales ha sido un trabajo reconfortante y enriquecedor. Gracias a él he desarrollado un mayor dominio del desarrollo mediante Node.js y Angular, el cual podrá ser de gran ayuda en mi futuro laboral.

Los objetivos iniciales del proyecto han sido alcanzados completamente, puesto que se ha conseguido proporcionar a los clientes una forma más sencilla de encontrar y conocer charangas. Se ha proporcionado a las charangas una forma más sencilla de conseguir nuevos contratos, una herramienta para anunciarse y mostrar disponibilidad, y una novedosa forma de encontrar refuerzos para sus actos. Finalmente, a los músicos se les ha proporcionado una forma de promoción y una herramienta que les permite encontrar charangas para realizar más actos cuando ellos lo deseen.

Por tanto, el desarrollo de esta aplicación para agrupaciones musicales puede suponer una significativa mejora para el gremio musical, ya que permite facilitar la tarea de las personas que realizan gestiones en este campo y en la gran mayoría de ocasiones lo realizan de forma altruista.

10.2. Posibles mejoras

En versiones futuras de la aplicación se pueden incorporar una serie de mejoras que pueden realizar la aplicación aún más atractiva para el público, estas mejoras podrían ser las siguientes:

- Incorporar un apartado de calendario. En este apartado aparecería un calendario o agenda, donde se especificarían los próximos actos contratados, de manera que el cliente, músico o charanga, vea de una simple vista sus actuaciones próximas.
- Incorporar un apartado para gestionar las charangas internamente. Este apartado estaría enfocado en optimizar y reunir todas las gestiones que se realizan dentro de la charanga, como por ejemplo: Gestión de repertorio, formación de plantillas dentro de la charanga, gestión de ensayos y participación del grupo.

- Incorporar un foro para que los usuarios interactúen entre ellos. Este foro estaría destinado a que las charangas debatan e intercambien repertorio, los músicos puedan conocerse entre ellos, de manera que también se fomenten valores como la amistad y el respeto.
- Internacionalización de la aplicación: La aplicación actualmente ha sido desarrollada para España, ya que solo puedes filtrar por provincias españolas. En un futuro se podría abrir al mundo y permitir que los músicos de todo el mundo puedan gestionar sus actos de una manera más sencilla.
- Abrir la aplicación a cualquier grupo musical: Para facilitar su desarrollo, la aplicación ha sido pensada para grupos musicales de tipo charanga, los cuales son grupos reducidos y cuentan con instrumentos de viento y percusión. En versiones futuras se podría abrir el abanico a cualquier grupo musical, añadiendo todos los instrumentos que existan, de manera que orquestas o grupos de flamenco, puedan gestionar también sus actuaciones musicales.

11. Referencias

Arlequin Band Street. (s.f.). Obtenido de <https://www.arlequinband.com/charanga/que-es-una-charanga.html>

Atlassian. (s.f.). Obtenido de <https://www.atlassian.com/es/agile/kanban/boards#:~:text=Un%20tablero%20de%20kanban%20es,orden%20de%20su%20trabajo%20diario>.

Axarnet. (s.f.). *Axarnet*. Obtenido de <https://axarnet.es/blog/angular>

Azure. (s.f.). Obtenido de <https://learn.microsoft.com/es-es/azure/azure-portal/azure-portal-overview>

B, G. (s.f.). *Hostinger*. Obtenido de <https://www.hostinger.com/es/tutoriales/que-es-github>

charangas.com. (s.f.). *charangas.com*. Obtenido de [charangas.com: https://charangas.com](https://charangas.com)

Charangas.info. (s.f.). *Charangas.info*. Obtenido de <https://www.charangas.info>

Cimas Cuadrado, G. (s.f.). *openwebinars*. Obtenido de <https://openwebinars.net/blog/que-es-visual-studio-code-y-que-ventajas-ofrece/>

Fernández, Y. (s.f.). *Xataka*. Obtenido de <https://www.xataka.com/basics/dall-e-3-que-como-funciona-sus-novedades-que-puedes-hacer-esta-inteligencia-artificial>

Gonçalves, M. J. (s.f.). *hiberus*. Obtenido de <https://www.hiberus.com/crecemos-contigo/que-es-angular-y-para-que-sirve/>

Izertis. (s.f.). *Izertis*. Obtenido de <https://www.izertis.com/es/-/blog/encryptacion-de-password-en-nodejs-y-mongodb-bcrypt>

kinsta. (s.f.). *kinsta*. Obtenido de <https://kinsta.com/es/base-de-conocimiento/que-es-express/#:~:text=js%3F-,Express,desarrollar%20aplicaciones%20de%20backend%20escalables>.

Microsoft. (s.f.). Obtenido de <https://learn.microsoft.com/es-es/azure/mysql/flexible-server/overview>

Microsoft. (s.f.). Obtenido de <https://learn.microsoft.com/es-es/azure/app-service/overview>

Microsoft. (s.f.). Obtenido de <https://learn.microsoft.com/es-es/azure/static-web-apps/overview>

Node.js. (s.f.). Obtenido de <https://nodejs.org/es>

OpenWebinars. (s.f.). Obtenido de <https://openwebinars.net/blog/que-es-mysql/>

OpenWebinars. (s.f.). Obtenido de <https://openwebinars.net/blog/que-es-json-web-token-y-como-funciona/>

Unir. (s.f.). *Unir*. Obtenido de [https://www.unir.net/revista/empresa/analisis-dafo/#:~:text=El%20análisis%20DAFO%20\(Debilidades%2C%20Amenazas,en%20sus%20productos%20o%20servicios.](https://www.unir.net/revista/empresa/analisis-dafo/#:~:text=El%20análisis%20DAFO%20(Debilidades%2C%20Amenazas,en%20sus%20productos%20o%20servicios.)

Zendesk. (s.f.). Obtenido de <https://www.zendesk.com.mx/blog/metodologia-agil-que-es/>